When Assurance Authority Changes: Reward Poisoning Across Learning-to-Deployment Transitions

***Abstract*—Runtime assurance can mask an unsafe raw policy during protected adaptation. We study update-data integrity across runtime-authority transitions: an attacker alters bounded reward records consumed by an updater, while runtime states, actions, code, parameters, and certification remain trusted. Evidence collected under resident predictive authority is then reused to authorize permanent raw release.**

On a third-party published shield-retirement lifecycle, bounded reward-log poisoning confined to the shielded bootstrap phase already makes the raw policy 1.6 to 3.6 times more dangerous at retirement on all three seeds, and clean post-retirement learning repairs the damage on only one. A controlled Safe-Control-Gym study with five untouched seeds yields 27/120 poisoned raw-release failures versus 2/120 clean and 0/120 under resident predictive authority. The attack is contract-specific and detectable by task-aware reward checks, and does not stably reproduce in the official Safe-Control-Gym PPO learner; on the third-party pipeline a separate PPO implementation shows a paired-significant post-retirement effect on one of three seeds, so the boundary is implementation- and training-state-sensitive. The attack does not establish benign utility over freeze.

I. Introduction

Runtime assurance changes what an observer learns about an adaptive controller. If a decision module replaces unsafe proposals with actions from a trusted baseline controller, every adaptation rollout may remain physically safe even while corrupted update data move the raw policy toward failure. This masking is benign while the decision module remains resident: the deployed object is the assured composition. It becomes a security problem when the runtime authority is *retired*, after which the learned policy acts without the decision module that made the adaptation evidence safe.

We study the same authority transition in two deployment patterns. In the first, a learning controller is *protected while commissioned* and then released as a permanent raw snapshot under an explicit split-trust precondition: an attacker can alter only bounded reward records consumed by an updater, while runtime states, actions, code, parameters, and certification remain trusted. In the second, an independently published workflow *bootstraps* a policy-gradient learner behind a runtime shield and later retires the shield's authority, comparing sudden and smooth removal. These patterns differ in origin but share one assumption that we question: that the update-data provenance used during protected adaptation remains valid for the authority transition.

The distinction is easy to blur because runtime-assurance systems already support online upgrades, forward switching, reverse switching, and policy repair. Simplex retains a decision module and assured baseline controller [1]; Neural Simplex adds reverse switching and online retraining [2]; and barrier-based Simplex derives switching conditions that tolerate an unsafe or adversarial advanced controller under bounded model error [3]. Runtime policy repair [4], repair with preservation [5], and safe policy projection [6] further constrain or repair learned policies. These works establish that resident assurance and trusted repair can be effective. They do not establish that finite evidence sufficient for a resident decision module remains sufficient after that module is removed, when the update log itself is adversarial.

We study that gap under an explicit split-trust precondition. Safety-critical runtime control and asynchronous learning-data services need not share an integrity domain; OT guidance treats data management and historians as assets that exchange information across control and business zones [7], [8]. Our benchmark starts *after* compromise of a reward-ingestion principal: the attacker may change only bounded scalar rewards consumed by a policy-gradient updater. It cannot write authenticated runtime states, actions, gradients, learner code, parameters, optimizer state, certificates, or deployment states. We do not claim to demonstrate the initial service exploit or to evade log-integrity controls. If reward is deterministically recomputable from authenticated transitions, trusted recomputation blocks the evaluated attack.

The attack instead asks what follows when this integrity precondition is absent. It steers the raw policy toward a target that passes a finite reverse-switch check at release time but fails later without the decision module. We call this a *certificate-evasive update*: it is evasive only with respect to the stated finite release contract, not to a reward monitor or a sound invariant certificate.

The attack channel has useful structure. For a fixed policy-gradient batch, bounded reward changes first form a centered reward-to-go zonotope. Because our learners standardize advantages, the resulting parameter update is *not* an affine zonotope: normalization maps the centered returns to the unit sphere before the policy-score map is applied. We derive this

Network and Distributed System Security (NDSS) Symposium 2027
22–26 March 2027, Seoul, Republic of Korea
Anonymous submission for review
www.ndss-symposium.org

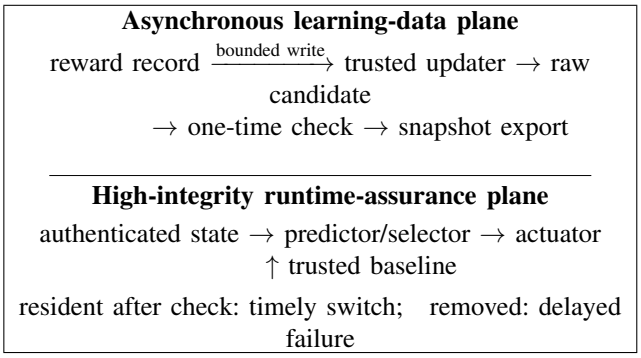

Fig. 1. Evaluated trust boundary. The benchmark begins after bounded reward-record access is obtained; it does not model the initial service compromise. Upstream provenance or recomputation removes this attack path, whereas resident predictive authority contains its downstream physical consequence. The same boundary is evaluated on a third-party shield-retirement workflow in Section IV.

normalized reward-influence set and use conic homogenization to remove the zero-vector degeneracy. Positive support against a linear certificate halfspace remains a second-order-cone feasibility problem under global gradient-norm clipping. Coordinatewise parameter clipping remains an explicit boundary. This gives an exact batch-audit bridge from reward corruption to release-gate margin, not a multi-batch attack optimizer. The analysis is supporting: it explains why a batch-local reward change can cross a release margin, but it neither constructs nor explains the multi-batch exploit, which remains the locked target-shaping rule.

A. Evidence Stack and Contributions

Section III-D formalizes the transfer boundary common to both studies: an evidence tuple $E = (s, a, p, i, \tau)$ (subject, authority, provenance, information, horizon) collected under the training contract must re-establish every component the deployment contract changes (Proposition 1). Our evidence combines a third-party case study with a controlled mechanism study. On the published shield-retirement workflow of Carr et al. [9], we run the official implementation unmodified except for a locked reward-record poisoning hook and metric instrumentation, and confine poisoning to the shielded bootstrap phase. The released raw policy is already 1.6–3.6$\times$ more dangerous at retirement on all three seeds, and clean post-retirement learning repairs the damage on only one. The same attack on the vendored tf-agents PPO learner of that lifecycle is paired-significant on one of three seeds (1178 $\rightarrow$ 2439 post-removal violations; McNemar exact $p = 2.6 \times 10^{-149}$), so the transfer-sensitive character is not specific to REINFORCE. This shows the authority-transition failure is not an artifact of a self-defined benchmark: an independently designed lifecycle that retires runtime authority inherits the update-data integrity assumption.

The controlled study then answers mechanistic questions on Safe-Control-Gym. After a sequential development block freezes the five-step check, target, bounded reward rule, detector, and stop rule, five untouched learner/ evaluation seed pairs are run once. All 120 poisoned snapshot–state pairs pass the initial check. Poisoned permanent raw release fails physically on 27/120, versus 2/120 paired clean failures; 25 poison-only versus zero clean-only discordances give two-sided exact $p = 5.960 \times 10^{-8}$. Pairs within a learner seed are not independent replicates, so five-seed reproduction, not the count, conveys generality. Every seed reproduces poisoned failure. Resident predictive authority switches before all 27 corresponding failures and incurs 0/120 failures. A locked sweep shows the separation at horizons 3, 5, and 10 rather than only the development-fixed horizon 5. An independent three-seed audit retains a one-step resident kernel as a negative control: it fails on 14/71 pairs and encounters 41 empty-kernel fallbacks, whereas resident predictive authority again incurs no failure. Thus recoverability and continued authority matter; residence alone is insufficient.

Broader Safe-Control-Gym evidence characterizes the conditional attack and its costs. Reward-log-only poisoning causes 38/188 delayed cartpole failures and 68/72 delayed two-dimensional quadrotor failures, compared with 1/188 and 0/72 for paired clean residual-REINFORCE learners. Commit admission and always-freeze have no observed failure. A unified quadrotor audit compares raw release, commit, freeze, a permanent backup shield, and the official linear MPSC on identical rollout blocks; it exposes different coverage, intervention, reward, and latency costs. Two locked negative gates remain central: the fixed-budget attack does not stably reproduce in the official proximal policy optimization (PPO) learner, and admitted updates do not outperform freeze under a benign actuator shift. The effective reward rule is also not stealthy: frozen simple log checks detect over 90% of poisoned batches at low false-positive rate, while trusted reward recomputation detects every edit.

Our contributions are:

- Under an explicit compromised reward-ingestion precondition, we identify a concrete assurance-contract vulnerability: reverse-switch evidence collected under protected adaptation does not transfer automatically to permanent raw release under adversarial online updates. We distinguish this claim explicitly from prior resident Simplex, shielded learning, policy repair, and safe-update work.
- We demonstrate the same authority-transition failure on a third-party published shield-retirement lifecycle (Carr et al. AAAI'23 official code): bounded reward-record poisoning confined to the shielded bootstrap phase already degrades the raw policy at retirement on 3/3 seeds, and the damage persists after clean post-retirement learning on 2/3.
- As supporting analysis, we derive the normalized reward-influence geometry of a standardized-advantage policy-gradient batch and an exact positive-support computation for linear release-gate halfspaces under global gradient-norm clipping, while excluding coordinatewise parameter clipping; this analysis explains batch-local influence and does not construct the multi-batch exploit.
- After locking a fixed-target reward attacker and detector,

we run five untouched confirmation seeds. Poisoned raw release yields 27/120 failures versus 2/120 clean and 0/120 under resident predictive authority; all five seeds reproduce failure, and three monitor horizons separate the contracts. An independent audit retains the failed one-step resident-kernel result, 14/71 versus 0/71 under that authority.
- We retain the broader two-system attack, contract-cost audits, and negative PPO and benign-utility results to delimit learner generality, sampled-certificate strength, and the practical cost of each sound escape condition.

II. Model and Problem Statement

A. Attacked Histories and Plausible States

We consider a discrete-time cyber-physical system

$$x_{t+1} = f(x_t, u_t, w_t), \qquad \widetilde{y}_t = h(x_t) + a_t, \tag{1}$$

where x_t is the physical state, u_t is the control input, $w_t \in \mathcal{W}$ is ordinary process uncertainty, and a_t is an FDI signal applied to the measurement channel. The controller observes an attacked history

$$h_t = (\widetilde{y}_{0:t}, u_{0:t-1}, \widetilde{\ell}_{0:t}), \tag{2}$$

where $\widetilde{\ell}_t = \ell_t + b_t$ denotes rewards, logs, or auxiliary signals used by the learning rule and b_t is update-data corruption. The two channels need not both be present. Sensor FDI a_t creates uncertainty about the physical state; log corruption b_t can redirect an update even when the state is trusted. Both matter to the certificate lifecycle because the update rule consumes h_t.

Following severe sensor-attack safety work [10], we do not assume that h_t identifies a unique physical state. For an attack family $\mathcal{A}$, define the plausible-state set

$$X_{\mathcal{A}}(h_t) = \{x : \exists E \in \mathcal{A},\ H_t(E) = h_t,\ x_t(E) = x\}. \tag{3}$$

This set is the semantic object over which any strategic-FDI-sound certificate must range. If a certificate is sound only for a nominal estimate $\hat{x}(h_t)$, but $X_{\mathcal{A}}(h_t)$ contains another compatible state, the certificate is not sound against the attack family.

B. Online-Updated Controllers

The controller is a history-dependent policy π_{θ_t} with parameters θ_t. It selects actions as $u_t = \pi_{\theta_t}(h_t)$ and updates according to

$$\theta_{t+1} = U(\theta_t, h_t). \tag{4}$$

We call an update nontrivial if it changes the controller's action map on some reachable or admissible history. Parameter changes that leave all actions unchanged are behaviorally trivial and do not count as learning in this paper.

C. Certificates and Certified Kernels

Let Γ denote a certificate type. In this paper, Γ can be an invariant set, a barrier certificate, a reachability certificate, or a runtime shield. For a history h, the certificate induces a set of actions that are certified for every plausible state:

$$K_\Gamma(h) = \{u \in \mathcal{U} : u \text{ satisfies } \Gamma \text{ for all } x \in X_{\mathcal{A}}(h)\}. \tag{5}$$

We call $K_\Gamma(h)$ the *certified kernel*. Existing severe-attack safety filters can be viewed as mechanisms for constructing and acting within such kernels for fixed controllers. Our focus is the additional gate required when U changes π_θ after deployment.

Definition 1 (Certified online learning)**.** *A mechanism is a certified online learner at history h if it accepts a certificate for the updated controller $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$, and claims that the next certified action is sound for every state in $X_{\mathcal{A}}(h)$.*

Definition 2 (Learning-freeze frontier)**.** *For a family of histories parameterized by attack strength or ambiguity radius, the learning-freeze frontier is the boundary at which $K_\Gamma(h)$ becomes empty, the candidate updated action leaves $K_\Gamma(h)$, or the candidate parameter leaves the corresponding certified set $\Theta_\Gamma(h)$. Beyond this boundary, a sound certified learner must reject the certificate, freeze the update, or obtain extra trusted state information.*

III. Threat Model

A. System Roles and Deployment Contracts

The defender deploys an online-updated controller π_{θ_t} and a certificate mechanism. A severe-attack-aware estimator or filter can return a plausible-state set $X_{\mathcal{A}}(h_t)$ and certified action kernel $K_\Gamma(h_t)$. The defender may also possess a trusted state anchor, backup controller, or external fail-safe, but each is an explicit architectural assumption and cost.

We distinguish two deployment contracts. Under *snapshot deployment*, a safety layer protects data collection and adaptation, after which a raw snapshot π_{θ^+} may be committed for deployment. The snapshot itself therefore requires admission. Under *permanently filtered deployment*, a sound filter F remains online and the certified object is the composition $F \circ \pi_{\theta^+}$. Our raw-snapshot attack does not bypass such a filter; we evaluate permanent filtering as an escape condition and measure its intervention and reward cost.

We use four contract names consistently. *Permanent raw release* removes runtime authority after a one-time five-step check. *Resident predictive authority* repeats that check online and can switch permanently to the trusted baseline. The *one-step resident kernel* is a current-state action filter retained as a negative control. *Commit admission* instead certifies a candidate snapshot on a declared state set and horizon before raw deployment. These labels identify evidence scope, not interchangeable algorithm names.

Permanent removal is an intentional handoff, not a crash of the safety layer. Examples include protected commissioning

TABLE I
EVALUATED WRITE BOUNDARY. "TRUSTED" MEANS OUTSIDE ATTACKER WRITE AUTHORITY, NOT FORMALLY VERIFIED.

Object	Defender assumption	Attacker access
Runtime state, action, plant	Authenticated control path	No write
Baseline, selector, certificate	Trusted runtime code/state	No write
Transitions and learner code	Trusted after collection	Read only
Reward record at ingestion	No origin/semantic check	Bounded scalar write
Parameters and optimizer state	Written only by updater	No write
Held-out states and release rule	Fixed, honest evaluation	No write or selection

followed by snapshot export, an edge learner promoted to a lower-latency control target, or a fleet update whose deployment target does not carry the training-time predictor. Operators may prefer this handoff to avoid permanent compute, intervention, or dependency on a backup service; Section VIII measures those costs. Our claim is conditional on making that handoff. If runtime authority remains resident, the raw-policy failure is contained rather than physically realized.

B. Split-Trust Learning Pipeline

We separate a high-integrity runtime-assurance plane from an asynchronous learning-data plane. This is a least-privilege model, not a claim that the entire controller is compromised. The runtime plane owns authenticated plant state, the action selector, actuator interface, trusted baseline, and certificate outcome. The learning plane consumes recorded transitions and a scalar reward through a telemetry, historian, replay-buffer, or external-KPI interface. OT guidance explicitly separates control from data-management assets and recognizes historians that exchange data across zones [7], [8]. These architectures motivate the split; our benchmark does not emulate compromise of a particular historian or product.

The evaluated attack begins after a principal with this reward-field write capability has been compromised. Establishing that foothold is outside the experiment. The model fits post-computation/pre-update mutation by a replay writer or broker. A compromised reward producer is equivalent only if it has the same bounded output authority; source authentication alone would then authenticate a malicious value rather than establish its semantics.

C. Attack Channels and Capabilities

The general history model permits two attack channels. Observation FDI a_t changes the state semantics of the history and enlarges $X_{\mathcal{A}}(h_t)$. Update-data corruption b_t changes rewards, logs, or auxiliary learning signals and can redirect $U(\theta_t, h_t)$ even when the state is trusted. The formal lifecycle condition applies to either channel because both can affect the next action map.

The end-to-end learner experiments deliberately use the narrower reward-log channel. The attacker is white-box and batch-aware: it observes the current learner and adaptation batch, but not the held-out deployment-state choice, and modifies only scalar rewards before an ordinary policy-gradient update, within a declared per-step L_∞ budget. It cannot write actuator commands, physical states, observations, dynamics, gradients, policy parameters, optimizer state, deployment states, or certificate outcomes. The adaptation selector sees the learner's proposed action before the plant step. Physical safety during collection therefore does not arise from giving the attacker direct policy-write or actuator access.

D. Evidence Transfer Across Authority Transitions

The contracts above differ in what an acceptance decision may rely on, so evidence valid under one contract need not be valid under another. We make that boundary explicit. An assurance decision consumes an *evidence tuple*

$$E = (s, a, p, i, \tau), \tag{6}$$

whose components are: the *subject* s, either the raw policy π_θ or the assured composition $F \circ \pi_\theta$; the *authority* a, either a resident decision module, shield, or filter that constrains applied actions, or no authority; the *provenance* p of the learning signals, trusted update data or an adversarial update log with bounded reward write access; the *information* i available to the subject at decision time (full state, partial observation, belief state, or the filtered action available only under the authority); and the *horizon* τ over which the evidence is claimed to bound behavior (finite evaluation window or permanent future). A *contract transition* $C_{\text{train}} \to C_{\text{deploy}}$ changes one or more of these components. Evidence E collected under C_{train} transfers to C_{deploy} only if each changed component is re-established under C_{deploy}:

- *subject*: evidence about the composition does not bound the raw policy, and a raw-policy audit does not certify the composition;
- *authority*: evidence collected while a resident module masked unsafe proposals describes the masked process, not the raw policy after the module is removed;
- *provenance*: a policy trained on an adversarial update log is not the output of the trusted process the evidence was computed on;
- *information*: behavior under the authority's information set does not determine behavior under the deployment observation;
- *horizon*: a finite evaluation pass does not bound an unbounded future.

Proposition 1 (Transfer obligation)**.** *Let E be evidence collected under contract C_{train}, and let the transition $C_{\text{train}} \to C_{\text{deploy}}$ change a nonempty subset of the components of the evidence tuple. Acceptance under C_{deploy} is unsound unless each changed component's evidence obligation is re-established under C_{deploy}.*

Proof. Each component is a premise of the evidence's soundness: the object the evidence is about, the enforcement that governed collection, the provenance of the learning signals,

the information available to the subject, and the horizon. Removing or weakening a premise does not preserve conclusions drawn from the original premise set. Permanent raw release, the contract defined above, changes all five components at once: subject from composition to raw policy, authority from resident to none, provenance from trusted to adversarial update log, information from masked action to deployment observation, and horizon from finite check to permanent future. Each change removes a premise of the finite reverse-switch evidence collected under resident authority, so that evidence cannot be reused unchanged. □

The related-work landscape and our experiments occupy distinct positions in this space. Hsu et al. [11] is the positive control: they precede authority removal with independent evidence about the raw policy in the deployment environment, re-establishing the changed components. Carr et al. [9] define the lifecycle we attack — a shield bootstrap that later retires its authority — and therefore realize the subject and authority transitions, but under trusted provenance and without an adversarial update log. Simplex and its descendants [1]–[3] never change the authority component: the certified object is the resident composition. Our snapshot release is authority removal under an adversarial update log; the third-party case study in Section IV instantiates the same transition on a published lifecycle, and the controlled study in Section VIII isolates it with matched clean and poisoned runs. The implication for defense is that each escape condition re-proves a different subset of the tuple, which is why the same attack shape must be re-verified against provenance, recomputation, resident authority, commit admission, and permanent filtering separately.

E. Integrity Assumptions and Escape Conditions

The vulnerable configuration supplies neither origin integrity for the reward record nor trusted reward recomputation. Binding a reward, transition, action, producer identity, and time to an authenticated append-only record blocks post-ingestion mutation. If reward is a deterministic function of authenticated transitions, recomputation by a trusted principal completely blocks the evaluated channel. If reward depends on delayed quality labels, operator feedback, or an external KPI unavailable to the runtime plane, the system instead needs a trusted producer, redundant validation, or a detector; a signature applied to an already malicious value is insufficient.

These controls answer different questions. Provenance and recomputation remove the upstream attack precondition. A task-aware anomaly detector may reject suspicious batches but is distribution- and semantics-dependent. A commit certificate does not sanitize the log; it limits which resulting snapshot can be released. Resident predictive authority likewise does not detect corruption, but can contain its physical consequence. Our frozen detection audit reports trusted recomputation at TPR/FPR $1.000/0$ and simple task-aware checks with TPR above 0.90 and FPR at most 0.036. The supported attack is therefore effective under the exposed interface but not stealthy against the tested checks.

F. Security and Availability Goals

We separate four attacker or failure objectives:

- an *unsafe certified action*, where the mechanism accepts an applied input that is unsafe for some state in $X_{\mathcal{A}}(h_t)$;
- a *stale certified policy*, where the current applied input is filtered but the accepted raw updated action map leaves the kernel;
- *uncertified learning*, where a nontrivial update continues after certificate rejection; and
- *forced freeze*, where the attacker expands the plausible set or redirects the update until sound learning must stop or re-anchor.

The defender additionally cares about availability: admitted state coverage, accepted update fraction, task reward, filter intervention and rejection, certificate latency, and trusted-anchor use. Zero physical violations with no benefit over always-freeze is safe but does not establish useful learning.

G. Non-goals

We do not assume that every attack makes safety impossible. Existing severe-attack filters, secure estimators, and fault-tolerant CBF methods can preserve safety when their assumptions make $K_\Gamma(h_t)$ nonempty [10], [12], [13]. We do not claim to defeat a permanently sound runtime filter, to prove continuous-state invariance from a sampled certificate, or to establish vulnerability for all learner families. The question is narrower: which certificate must cover an update, which escape conditions remain sound, and what coverage and availability costs follow? We additionally do not claim a zero-day against a logging service, reward-monitor evasion, or persistence when authenticated trusted recomputation is available.

IV. Third-Party Shield-Retirement Case Study

The controlled contract in Section VIII is defined by this paper. To test whether the authority-transition failure is an artifact of a self-defined benchmark, we repeat the same security question on an independently published lifecycle that retires runtime authority. Carr et al. [9] (AAAI 2023) train a policy-gradient agent for 1000 episodes behind a belief-support shield computed by Storm on a partially observable gridworld, then either suddenly or smoothly switch the shield off, and compare the released policy against retained-shield and no-shield controls. We use their official public implementation and change only the learner-visible reward records: the attacker writes $r' = r + \xi$ with $|\xi| \le \delta$ per record into the update log, and touches no other plane. States, observations, shield semantics, environment dynamics, learner code, hyperparameters, and evaluation are the upstream originals; every patch is itemized in the artifact. Training is 5000 episodes for REINFORCE and 10^5 environment steps for PPO, and retirement is sudden at episode 1000 (REINFORCE) or env-step 10^5 (PPO), following the upstream protocol.

a) Fidelity gate.: On the obstacle domain (N=6), the clean condition reproduces the qualitative ordering of Carr et al.'s Table 1 for REINFORCE: shield retained 0/0 violations during/after training, smooth 326/675, sudden 3277/3784, and

TABLE II
RETIREMENT-BOUNDARY CAUSAL ISOLATION ON THE THIRD-PARTY LIFECYCLE (OBSTACLE, REINFORCE, SUDDEN). POISONING IS CONFINED TO THE SHIELDED BOOTSTRAP PHASE AND STOPS AT RETIREMENT; ALL LATER TRAINING IS CLEAN. AT-RETIREMENT IS THE RAW POLICY EVALUATED AT THE SWITCH INSTANT (1000 EPISODES); FINAL IS 5000 EPISODES. RATES ARE VIOLATION FRACTIONS.

	at retirement		final	
seed	clean	poisoned	clean	poisoned
1	0.216	0.777	0.236	0.235
2	0.484	0.763	0.234	0.751
3	≈0.24	0.748	0.233	0.741

no-shield 3798/3784. Shield $<$ smooth $\ll$ sudden $\leq$ no-shield is the ordering the authors report, so the victim workflow is not distorted by our patch.

b) Causal isolation at the retirement boundary.: Under a locked protocol (artifact amendment v1.3), poisoning is confined to the shielded bootstrap phase: the reward-record mutation is active only while the shield enforces the action mask and stops exactly at the retirement instant, so all subsequent learning is clean. We evaluate the raw policy at the switch instant before any unshielded learning (1000 episodes) and again after the full training budget (5000 episodes), on paired seeds. Table II and Fig. 2 show the result.

On all three seeds the poisoned raw policy is already 1.6–3.6$\times$ more dangerous at the retirement instant than its clean paired control, while physical violations during the protected phase remain zero in every run. The damage is latent in the policy *before* any unshielded learning, and it is created entirely inside the shielded bootstrap phase. After 4000 episodes of fully clean post-retirement training the damage persists on two of three seeds (0.751 and 0.741 vs clean 0.234 and 0.233); one seed self-heals (0.235). The mechanism signal is the unmasked raw-policy/shield disagreement during the protected phase, which stays near 0.01–0.05 for clean runs but rises to 0.11–0.15 (peak 0.71) under poisoning while masked violations remain zero: the shield masks unsafe proposals whose physical consequence appears only after authority removal (Fig. 3).

c) Learner boundary.: The same contrast attack (δ=2) on the third-party PPO pipeline (vendored tf-agents PPO with action masking, 10^5 env steps, sudden retirement) raises post-removal violations on one of three seeds (1178 $\rightarrow$ 2439; McNemar exact $p = 2.6 \times 10^{-149}$; first-violation episode $6 \rightarrow 0$), while the other two seeds show no change ($p = 1.00$ and 0.96). The vulnerability is therefore real and paired-significant but transfer-sensitive: it is not a universal learner property, and the effect concentrates on seeds whose clean learning would otherwise transfer safety to the released policy.

d) Attack budget and susceptibility.: On the full-phase REINFORCE protocol the effect is budget- and state-dependent (paired seeds within the same locked code version). On the transfer-sensitive seed 2, δ=2 already saturates the effect: post-removal violations rise 1127 $\rightarrow$ 3762 (3.3$\times$; McNemar exact $p \approx 0$), and δ=10 gives 3766 (no further change). On seed 3 the effect requires a larger budget: δ=2 leaves violations unchanged (2482 vs. clean 2471; $p = 0.84$) while δ=10 raises them to 3700 ($p = 8.4 \times 10^{-138}$). The seed whose clean released policy is already in the unsafe regime (seed 1 at 75.7%) shows no additional damage at either budget; the bounded additive distortion is absorbed by the value baseline and return normalization. Negative controls delimit the attack shape: the bias variant is protective on the ceiling seed, and the risk variant is protective on seeds 1 and 3 but raises violations on the transfer-sensitive seed 2 (1127 $\rightarrow$ 2457, 2.2$\times$, $p = 2.0 \times 10^{-162}$), strictly weaker than the contrast attack on the same seed. The attack is thus not a monotone "more corruption $\Rightarrow$ more damage" effect; it is a shape-, budget-, and transfer-state-dependent steering of the released policy.

e) Reading.: The third-party workflow is not a synthetic victim: its shield bootstrap and retirement schedule are the authors' design, and the release-time raw policy inherits exactly the update-data provenance assumption the paper challenges. The security contribution of this paper is the integrity dimension on top of that published lifecycle, not the lifecycle itself. Absolute rates differ across instrumented and non-instrumented code versions because the unmasked-disagreement probe requires an independent belief tracker; all clean-vs-poisoned comparisons above are paired within a locked code version and are unaffected by that version change.

V. ASSURANCE-CONTRACT FAILURE MODES

This section separates three failure modes that are often conflated.

Nominal-state certification. A nominal certificate checks safety only at an estimate $\hat{x}(h)$ or at the reported sensor value. Under FDI, this can accept an action that is safe for $\hat{x}(h)$ but unsafe for another state in $X_{\mathcal{A}}(h)$. This is the classical state-semantics failure already addressed by severe-attack safety filters.

Action-only filtering. A robust safety filter can repair the current action by projecting it into $K_\Gamma(h)$. This protects the physical action applied at that step when the kernel is nonempty. It does not, by itself, certify the learner update. If the update rule still changes θ_t to θ_{t+1} such that the raw policy $\pi_{\theta_{t+1}}$ leaves $K_\Gamma(h)$, then the certificate applies to the filtered action, not to the updated learner. The next certificate must therefore reason about the updated action map, not only about the one action that was just projected.

Uncertified learning after rejection. When $K_\Gamma(h) = \emptyset$, a severe-attack filter can reject or enter fail-safe mode. If the learner nevertheless updates θ_t using the same attacked history, this update is not certified learning. The mechanism may still operate as an engineering fail-safe, but it cannot claim that the online learner remained inside a sound certificate lifecycle.

Protected adaptation, unsafe deployment. An adaptation shield can keep every data-collection transition safe while an attacker corrupts only the log consumed by the learner. If the resulting snapshot is later deployed without the shield, the attack succeeds only through the legitimate update rule. This

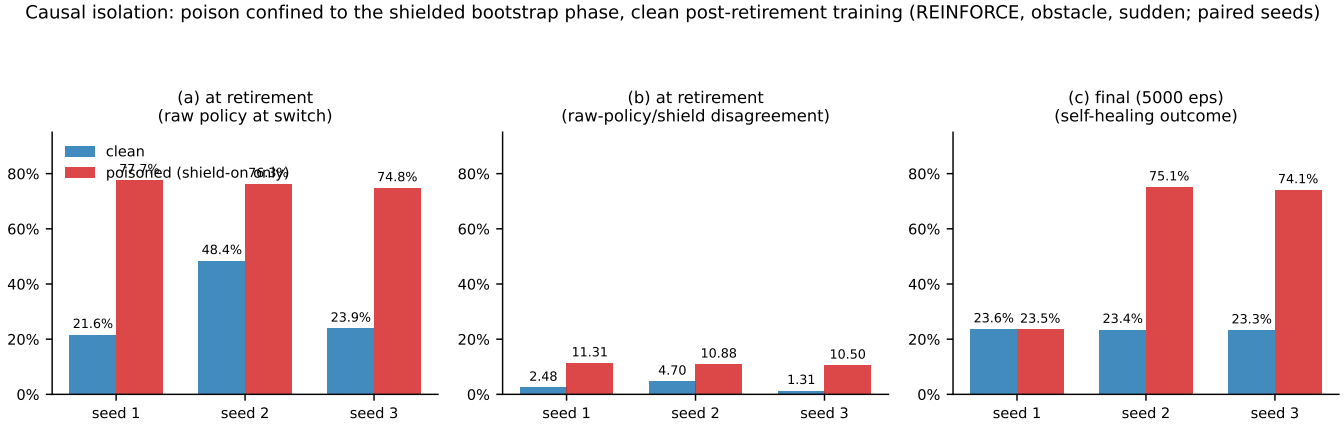

Fig. 2. Causal isolation on the third-party lifecycle (REINFORCE, obstacle, sudden, paired seeds). (a) At-retirement violation fraction of the raw policy; (b) at-retirement raw-policy/shield disagreement; (c) final violation fraction after clean post-retirement learning. Poisoning is confined to the shielded bootstrap phase.

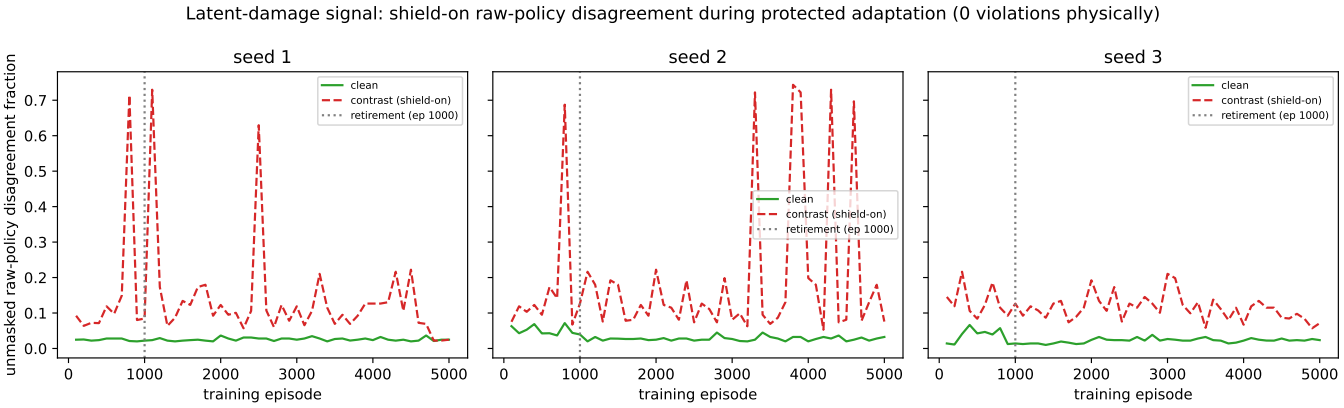

Fig. 3. Latent-damage signal during protected adaptation: unmasked raw-policy/shield disagreement rises under poisoning while physical violations remain zero, so the damage is masked until authority removal.

delayed failure is distinct from evading the runtime shield. A sound snapshot lifecycle therefore requires a commit certificate before the protected adaptation environment releases a raw policy.

These attacks motivate separate metrics. An *unsafe certified action* counts accepted certificates whose applied action is unsafe for some plausible state. A *stale certified policy* counts accepted certificates for which the updated raw policy leaves the kernel. *Uncertified learning* counts nontrivial updates after certificate rejection. End-to-end studies add delayed deployment violations, admission coverage, paired clean/poison outcomes, update fraction, intervention, reward, and certificate runtime.

VI. Contract-Aware Update Analysis

A. Current- and Future-History Gates

The central condition has two levels. The first is a current-history condition: after an update, the action selected at the history used for the update must still be certified. The second is a lifecycle condition: the updated action map must remain certified over the future attacked histories that can be reached after accepting the update.

Lemma 1 (Current-history gate)**.** *Let $M = (\pi_\theta, U, \mathsf{Cert})$ be a history-dependent online controller. For an attacked history h, let $X_{\mathcal{A}}(h)$ be the plausible-state set and let $K_\Gamma(h)$ be the certified kernel induced by certificate type Γ. If M accepts a sound certificate for the updated controller $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$, then*

$$\pi_{\theta_{t+1}}(h) \in K_\Gamma(h). \tag{7}$$

Consequently, if $K_\Gamma(h) = \emptyset$, then M cannot both accept the certificate and allow a nontrivial online update at h.

Proof. Soundness of the certificate means that the certified action must satisfy the certificate condition for every physical state that is compatible with the attacked history and the attack model. By definition, that set of states is $X_{\mathcal{A}}(h)$, and the set of actions satisfying the certificate condition for all such states is $K_\Gamma(h)$. If the updated controller remains in certified operation after the update, its selected action at h is $\pi_{\theta_{t+1}}(h)$

and must therefore lie in $K_\Gamma(h)$. If $K_\Gamma(h) = \emptyset$, no such certified action exists. A sound mechanism must then reject the certificate, freeze or roll back the update, or use additional trusted information that changes the plausible-state set and hence the kernel. □

Proposition 2 (Action filtering is not update certification)**.** *Let $F(v, h)$ be an action filter that returns an applied input in $K_\Gamma(h)$ whenever the kernel is nonempty. Consider a mechanism that first computes a candidate update $\theta^+ = U(\theta_t, h)$, then applies*

$$u_t = F(\pi_{\theta^+}(h), h), \tag{8}$$

but accepts the update without constraining the raw updated policy π_{θ^+}. If $\pi_{\theta^+}(h) \notin K_\Gamma(h)$, then the applied input can be certified while the updated learner is not certified at h. Consequently, action-only filtering does not imply the current-history gate in Lemma 1.

Proof. The filter property gives $u_t \in K_\Gamma(h)$, so the actuator input applied at the current step can satisfy the certificate. Lemma 1, however, is a condition on the updated controller claimed to remain in certified operation: it requires $\pi_{\theta^+}(h) \in K_\Gamma(h)$. When $\pi_{\theta^+}(h) \notin K_\Gamma(h)$, the certificate covers the filtered actuator channel, not the raw action map produced by the updated learner. Accepting the learner update under the same certificate therefore creates a stale certified policy. □

The lemma and proposition are local gates. They are enough to expose the stale-policy failure in a one-step study, but a controller whose parameters define actions at many future histories needs a stronger object. Let $\mathcal{H}_\mathcal{A}(h, \theta^+)$ be the set of attacked histories that can arise after history h when the updated controller π_{θ^+} is used and the attacker remains inside $\mathcal{A}$. Define the certified parameter set

$$\Theta_\Gamma(h) = \{\theta : \forall h' \in \mathcal{H}_\mathcal{A}(h, \theta), \pi_\theta(h') \in K_\Gamma(h')\}. \tag{9}$$

Theorem 1 (Future-history lifecycle gate)**.** *Let $\theta^+ = U(\theta_t, h)$ be a candidate online update. If M accepts a sound lifecycle certificate for π_{θ^+} from history h over the attack family $\mathcal{A}$, then*

$$\theta^+ \in \Theta_\Gamma(h), \tag{10}$$

equivalently,

$$\forall h' \in \mathcal{H}_\mathcal{A}(h, \theta^+), \quad \pi_{\theta^+}(h') \in K_\Gamma(h'). \tag{11}$$

If there exists $h' \in \mathcal{H}_\mathcal{A}(h, \theta^+)$ such that $\pi_{\theta^+}(h') \notin K_\Gamma(h')$, then accepting the update creates a stale certified policy. If some reachable h' has $K_\Gamma(h') = \emptyset$, certified learning cannot continue through that history without rejecting, freezing, rolling back, or obtaining trusted information that changes $X_\mathcal{A}(h')$.

Proof. A lifecycle certificate claims sound certified operation not only at the update history h, but at every attacked history h' reachable under the updated controller and the admissible attack family. Applying the soundness argument from Lemma 1 pointwise to each such h' gives $\pi_{\theta^+}(h') \in K_\Gamma(h')$. This is exactly the definition of $\theta^+ \in \Theta_\Gamma(h)$. If membership fails for any reachable history, the certificate no longer certifies the updated action map there; if the kernel is empty, no history-dependent action can satisfy the certificate at that history without changing the information set or rejecting certified operation. □

Proposition 3 (Linear-policy parameter gate)**.** *Consider a finite abstraction of future attacked histories indexed by observations $z \in \mathcal{Z}$, and a linear policy $\pi_\theta(z) = \theta^\top \phi(z)$. If each future-history certificate produces an interval kernel $K_\Gamma(z) = [\ell_z, u_z]$, then the lifecycle condition over $\mathcal{Z}$ is equivalent to the halfspace system*

$$\theta^\top \phi(z) \le u_z, \qquad -\theta^\top \phi(z) \le -\ell_z, \qquad \forall z \in \mathcal{Z}. \tag{12}$$

With optional box constraints on θ, the certified update set is a convex polytope. Rejecting candidates outside this set or projecting them onto it is sound for the finite abstraction.

Proof. For a fixed z, membership $\pi_\theta(z) \in [\ell_z, u_z]$ is exactly the pair of linear inequalities above. Enforcing this pair for every $z \in \mathcal{Z}$ is therefore equivalent to enforcing the future-history lifecycle gate on the abstraction. Intersecting these halfspaces with parameter box constraints preserves convexity. A candidate accepted after a successful feasibility check or projection satisfies every inequality, so its induced action lies in every certified interval kernel in the abstraction. □

B. Normalized Reward Influence on a Release Gate

We now connect the reward-log attacker to the halfspaces above. Consider a fixed on-policy batch of length T. Let $S \in \mathbb{R}^{T \times d}$ contain the policy-score row vectors, let $r \in \mathbb{R}^T$ be the logged rewards, and let $H_{ij} = \mathbf{1}[j \ge i]$ be the reward-to-go operator. Define the centering matrix

$$C = I - \frac{1}{T}\mathbf{1}\mathbf{1}^\top \tag{13}$$

and let the attacker choose $\delta \in [-\epsilon, \epsilon]^T$. The centered return vector is

$$y(\delta) = CH(r + \delta). \tag{14}$$

The fixed-batch assumption is appropriate for a reward-log-only attacker: changing a reward after collection does not change the states, actions, or score rows already in the batch.

Proposition 4 (Standardized reward-influence set)**.** *Suppose the learner standardizes the centered reward-to-go advantages with population standard deviation whenever $\|y(\delta)\|_2 > 0$. Before optimizer clipping, a gradient-ascent proposal of size α is*

$$\widetilde{\theta}^+(\delta) = \theta + \frac{\alpha}{\sqrt{T}} S^\top \frac{y(\delta)}{\|y(\delta)\|_2}. \tag{15}$$

Consequently, the raw one-batch proposal set is the image of the centered reward-to-go zonotope under normalization and the policy-score map; in general, it is not a parameter-space zonotope.

Proof. The standardized advantage vector is $y/\sqrt{T^{-1}\|y\|_2^2} = \sqrt{T}\, y/\|y\|_2$. Substitution into the usual mean score-function

gradient $T^{-1}S^\top(\sqrt{T}\,y/\|y\|_2)$ gives (15). Equation (14) is affine in the reward-box variable, but division by its Euclidean norm is not; hence the normalized image is generally non-affine. □

Theorem 2 (Positive support under global gradient clipping)**.** *Let $B = S^\top/\sqrt{T}$, let the learner cap the global gradient norm at $G > 0$, and assume coordinatewise parameter clipping is inactive over the reward box. The post-cap update before parameter clipping is*

$$\theta_G^+(y) = \theta + \frac{\alpha B y}{\max\{\|y\|_2, \|By\|_2/G\}}. \tag{16}$$

For a release halfspace $a^\top P\theta \le b$, define $q = \alpha B^\top P^\top a$, $y_0 = CHr$, and $M = CH$. For any fixed $t > 0$, a reward perturbation whose gate-coordinate increment is at least t exists if and only if the following second-order-cone problem is feasible:

$$\begin{aligned} &\lambda \ge 0, \qquad -\epsilon\lambda\mathbf{1} \le v \le \epsilon\lambda\mathbf{1}, \\ &\bar{y} = \lambda y_0 + Mv, \qquad q^\top \bar{y} = 1, \\ &t\|\bar{y}\|_2 \le 1, \qquad (t/G)\|B\bar{y}\|_2 \le 1. \end{aligned} \tag{17}$$

Feasibility is monotone in t, so bisection on $[0, \min\{\|q\|_2, \alpha G\|P^\top a\|_2\}]$ computes the exact positive support

$$\max\left\{0, \sup_{\|\delta\|_\infty \le \epsilon} \frac{q^\top y(\delta)}{\max\{\|y(\delta)\|_2, \|By(\delta)\|_2/G\}}\right\}. \tag{18}$$

The construction remains valid when the centered-return zonotope contains the origin; if $G = \infty$, the second norm constraint is omitted. If this support does not exceed the current halfspace margin, no admissible reward edit can cross that halfspace in the batch.

Proof. Global norm clipping gives the stated maximum in the denominator. The ratio is positively homogeneous, so optimization over the zonotope $Z = \{y_0 + M\delta : \|\delta\|_\infty \le \epsilon\}$ is equivalent to optimization over its conic hull. The first line of (17) is the perspective representation of that hull: $v = \lambda\delta$. This is the standard homogenization used in fractional and conic optimization [14], [15].

If a positive ratio reaches t, rescale its conic representative until $q^\top \bar{y} = 1$; its two denominator branches are then exactly the last line of (17). Conversely, any feasible $\bar{y}$ is nonzero because of the unit-numerator equality and recovers a reward-box ray whose ratio reaches t. Thus the origin cannot create the degenerate feasibility present in a direct zonotope formulation. Each fixed-t problem is a second-order feasibility-cone problem [16]; monotonicity permits quasiconvex bisection [17]. Cauchy–Schwarz supplies both stated upper bounds. The margin result follows by adding the maximum increment to the current gate coordinate. □

This theorem is deliberately batch-local. Later batches change S, r, and the visited states; accumulated attacks compose state-dependent normalized updates rather than one fixed convex set. Theorem 2 covers global Euclidean gradient clipping but not coordinatewise parameter clipping. The implementation also skips standardization below a numerical tolerance; Section VIII requires each reported cone witness to lie above that tolerance. The theorem characterizes batch-local authority, not a multi-batch attack optimizer.

C. A Margin-Robust Finite-Abstraction Lift

Proposition 3 is exact on its listed grid. Lifting that result to continuous reachable histories requires coverage and regularity assumptions. Let $z = \psi(h')$ be a finite-dimensional representation on which the updated policy depends. Suppose the fixed-controller certificate can be written as finitely many obligations

$$c_j(x, u, w, x^+) \le 0, \qquad j = 1, \dots, m, \tag{19}$$

where $x^+ = f(x, u, w)$. Invariant, discrete-time barrier, and bounded-horizon reachability checks can be written in this form after augmenting the state with time or certificate memory.

Theorem 3 (Margin-robust finite-abstraction sufficiency)**.** *Fix a candidate snapshot π_θ and a future-history horizon. Let $\{(z_i, x_{iq}, w_{ir})\}$ be a finite set such that, for every reachable attacked history h', every $x \in X_{\mathcal{A}}(h')$, and every $w \in \mathcal{W}$, some sampled triple satisfies*

$$\|z - z_i\| \le \delta_z, \quad \|x - x_{iq}\| \le \delta_x, \quad \|w - w_{ir}\| \le \delta_w, \tag{20}$$

where $z = \psi(h')$. Assume:

1) *$\pi_\theta(z) \in \mathcal{U}$ and $\|\pi_\theta(z) - \pi_\theta(z')\| \le L_\pi\|z - z'\|$;*
2) *the certificate model $\widehat{f}$ has uniform error $\|f(x, u, w) - \widehat{f}(x, u, w)\| \le \varepsilon_f$;*
3) *$\widehat{f}$ is separately Lipschitz with constants L_f^x, L_f^u, L_f^w; and*
4) *each c_j is separately Lipschitz in (x, u, w, x^+) with constants $L_j^x, L_j^u, L_j^w, L_j^+$.*

Define

$$\begin{aligned} \eta_j = {} & (L_j^x + L_j^+ L_f^x)\delta_x \\ & + (L_j^u + L_j^+ L_f^u)L_\pi\delta_z \\ & + (L_j^w + L_j^+ L_f^w)\delta_w + L_j^+\varepsilon_f. \end{aligned} \tag{21}$$

If every sampled triple satisfies

$$c_j\Big(x_{iq}, \pi_\theta(z_i), w_{ir}, \widehat{f}(x_{iq}, \pi_\theta(z_i), w_{ir})\Big) \le -\eta_j \tag{22}$$

for all j, then $\pi_\theta(\psi(h')) \in K_\Gamma(h')$ for every reachable attacked history h' in the horizon. Hence the snapshot satisfies the continuous future-history lifecycle condition of Theorem 1.

Proof. Fix h', $x \in X_{\mathcal{A}}(h')$, and $w \in \mathcal{W}$, and select a sampled triple using (20). Write $u = \pi_\theta(z)$ and $u_i = \pi_\theta(z_i)$. Policy regularity gives $\|u - u_i\| \le L_\pi\delta_z$. The model-error and Lipschitz assumptions give

$$\begin{aligned} \|f(x, u, w) - \widehat{f}(x_{iq}, u_i, w_{ir})\| \le {} & \varepsilon_f + L_f^x\delta_x \\ & + L_f^u L_\pi\delta_z + L_f^w\delta_w. \end{aligned} \tag{23}$$

Applying the four Lipschitz bounds of c_j and collecting terms yields

$$\begin{aligned} c_j(x,u,w,f(x,u,w)) &\le c_j(x_{iq},u_i,w_{ir}, \\ &\qquad \widehat{f}(x_{iq},u_i,w_{ir})) + \eta_j \\ &\le 0. \end{aligned} \tag{24}$$

The argument holds for every plausible state, disturbance, certificate obligation, and reachable attacked history, which is precisely membership in the corresponding kernels over the stated horizon. □

Theorem 3 is a conditional verification result, not a reachable-set construction. Its joint-cover premise must itself be justified; a finite rollout sample does not establish it. The coverage audit in Section VIII empirically probes this missing premise but does not replace a cover proof.

Proposition 5 (Plausible-set and freeze-frontier monotonicity)**.** *Fix a history domain $\mathcal{H}$, dynamics, certificate obligations, and policy class. If two information contracts satisfy $X_1(h) \subseteq X_2(h)$ for every $h \in \mathcal{H}$, let $K_{\Gamma,k}$ be the kernel induced by X_k and let $\Theta_{\Gamma,k} = \{\theta : \pi_\theta(h) \in K_{\Gamma,k}(h),\ \forall h \in \mathcal{H}\}$. Then*

$$K_{\Gamma,2}(h) \subseteq K_{\Gamma,1}(h) \quad \textit{and} \quad \Theta_{\Gamma,2} \subseteq \Theta_{\Gamma,1}. \tag{25}$$

Consequently, for a nested ambiguity family $X_{\rho_1}(h) \subseteq X_{\rho_2}(h)$ whenever $\rho_1 \le \rho_2$, certified update feasibility is non-increasing in ρ: once the certified parameter set is empty, it remains empty at larger ambiguity. Conversely, a trusted anchor that pointwise shrinks the plausible-state sets can only weakly enlarge the kernels and certified parameter set.

Proof. An action certified for every state in the larger set $X_2(h)$ is certified for every state in its subset $X_1(h)$, proving kernel inclusion. Requiring a policy action to lie in the smaller kernel $K_{\Gamma,2}(h)$ for every $h \in \mathcal{H}$ is at least as restrictive as requiring membership in $K_{\Gamma,1}(h)$, proving parameter-set inclusion. The frontier and anchor statements follow by applying these inclusions to nested families. □

The monotonicity comparison holds on a fixed history domain and certificate contract. If an anchor changes which histories are reachable, or if two sampled plausible sets are not nested, neither conclusion follows automatically. The P3 anchor audit therefore constructs nested samples explicitly.

The future-history theorem is a necessary certificate condition; Proposition 3 is exact only on its abstraction; and Theorem 3 supplies a sufficient lift only under its stated cover, regularity, and margin assumptions. None is a complete controller synthesis procedure or a claim that safety under sensor attacks is impossible.

D. Instantiating the Kernel

For an invariant certificate $I \subseteq S$, the kernel is

$$\begin{aligned} K_I(h) = \{u \in \mathcal{U} : \forall x \in X_{\mathcal{A}}(h), \forall w \in \mathcal{W}, \\ f(x,u,w) \in I\}. \end{aligned} \tag{26}$$

For a barrier certificate $B(x) \ge 0$ with safe set $C = \{x : B(x) \ge 0\}$, the corresponding kernel is

$$\begin{aligned} K_B(h) = \{u \in \mathcal{U} : \forall x \in X_{\mathcal{A}}(h), \forall w \in \mathcal{W}, \\ B(f(x,u,w)) \ge 0\}. \end{aligned} \tag{27}$$

For a reachability certificate, the initial set of the reachability computation must be $X_{\mathcal{A}}(h)$ rather than a nominal state estimate. For a runtime shield, the shield output must lie in $K_\Gamma(h)$; if the kernel is empty, the shield can only reject, enter fail-safe mode, or request a trusted state anchor.

E. A Minimal Counterexample

Consider

$$x_{t+1} = \lambda x_t + u_t, \qquad |u_t| \le u_{\max}, \qquad S = [-1,1]. \tag{28}$$

Suppose FDI makes the same history compatible with $x_t = r$ and $x_t = -r$, where $r \in (0,1]$ and $\delta = \lambda r$. The one-step safe action set for a state x is

$$\mathcal{U}_{\text{safe}}(x) = [-u_{\max}, u_{\max}] \cap [-1-\lambda x, 1-\lambda x]. \tag{29}$$

For $x = r$ and $x = -r$, these sets are individually nonempty but disjoint whenever

$$1 < \lambda r < 1 + u_{\max}. \tag{30}$$

The strict upper bound ensures that each state is individually controllable if it is known. The failure comes from FDI-induced indistinguishability: no single history-dependent action can certify both states.

This example distinguishes fixed filtering from certified learning. A fixed severe-attack filter can reject because the common kernel is empty. A certified online learner must do more: it must also prevent an update from continuing under a certificate that no longer has a feasible kernel.

F. Certificate-Gated Update

The theorem suggests the following update rule. First compute or obtain $X_{\mathcal{A}}(h)$ using a severe-attack-aware estimator or safety filter. Then compute $K_\Gamma(h)$. If the kernel is empty, reject the certificate and freeze the update. If the kernel is nonempty, apply the candidate update only if the updated action remains inside the kernel; otherwise project, constrain, or reject the update.

This rule should not be read as a complete controller synthesis method. Its role is to separate the fixed-controller safety problem, which prior work addresses, from the lifecycle condition needed when a controller learns from attacked histories.

VII. Deployment Contracts and Escape Conditions

The analysis supports several deployment responses rather than a single new filter. Our LifecycleGate prototype is a contract-enforcement layer around existing severe-attack-aware filters: it does not construct a new CBF or reachability certificate, but uses the kernel produced by such a certificate to

decide whether a learning update can be accepted, projected, or rejected.

Update-admission interface. At history h_t, an estimator supplies $X_{\mathcal{A}}(h_t)$, a certificate module supplies $K_\Gamma(h_t)$, and the learner proposes $\theta^+ = U(\theta_t, h_t)$. The prototype checks the current action against K_Γ or, when a future-history abstraction is available, checks or projects the full update against Θ_Γ. For linear policies this is the convex halfspace system in Proposition 3. A grid certifies only its listed points unless the cover, regularity, and model-error premises of Theorem 3 justify its robust margin.

Contract choices. The mechanism may accept, project, freeze or roll back, or request a trusted anchor that shrinks $X_{\mathcal{A}}$. Snapshot commit certifies a candidate on a declared state set and horizon, then backtracks along the connected segment from the last admitted snapshot so that a disconnected safe island is not accepted. A permanent filter must instead certify $F \circ \pi_{\theta^+}$ at runtime; resident predictive authority repeatedly tests recoverability and switches to the baseline. Commit exposes coverage and model-mismatch risk, whereas resident mechanisms expose intervention, rejection, and online cost.

Fail-closed semantics. An empty plausible-state kernel, an uncertified current snapshot, model failure, or solver failure cannot silently inherit the previous certificate. The gate freezes or rolls back and reports the failure. A trusted anchor may resume learning only by changing the justified plausible-state set; it is not modeled as a free oracle.

Utility is an independent property. The gate enforces certificate membership, not improvement of the task objective. A safe update can be neutral or harmful. Consequently, accepted parameter distance is an availability diagnostic but not proof of learning utility; utility requires paired task outcomes against always-freeze under a benign change.

VIII. Controlled Mechanism Study

Section IV established the authority-transition failure on an independently published lifecycle. The studies in this section are a controlled mechanism study on Safe-Control-Gym [18]: fully controlled contracts, paired comparisons, and mechanism audits answer five questions. (Q1) Does the same finite reverse-switch evidence support permanent raw release and resident predictive authority? (Q2) How does a bounded reward box influence release-gate halfspaces? (Q3) Can corruption limited to reward records cause delayed physical failure across systems? (Q4) Which reward-integrity assumptions remove that attack? (Q5) Which escape contracts prevent failure, at what cost, and with which generality limits? The first question is the direct mechanism behind the case study: the third party retires authority over a raw snapshot; here the same transfer is isolated with matched clean and poisoned runs, horizon sweeps, and an independent one-step negative control.

A. Setup

We instantiate the system in Section VI with $\lambda = 1.2$, $u_{\max} = 0.2$, and $S = [-1, 1]$. The FDI-induced ambiguity set is $X_{\mathcal{A}}(h) = [-\rho, \rho]$. The common certified kernel for the invariant certificate $I = S$ is

$$K(\rho) = [-u_{\max}, u_{\max}] \cap [-1 + \lambda\rho, 1 - \lambda\rho]. \tag{31}$$

The kernel is nonempty exactly when $\lambda\rho \le 1$.

TABLE III
Certified kernel for the one-dimensional ambiguity set.

ρ	$\lambda\rho$	$K(\rho)$	Gate
0.600	0.720	$[-0.200, 0.200]$	allow
0.800	0.960	$[-0.040, 0.040]$	constrain
0.833	1.000	$\{0\}$	boundary
0.900	1.080	$\emptyset$	freeze/reject
1.000	1.200	$\emptyset$	freeze/reject

B. Learning-Freeze Frontier

The frontier occurs at $\rho = 1/\lambda = 0.833$. Table III shows the exact kernel values for representative ambiguity radii.

At $\rho = 0.9$, the states $+0.9$ and -0.9 are each controllable if their sign is trusted. The safe intervals are $[-0.2, -0.08]$ for $+0.9$ and $[0.08, 0.2]$ for -0.9. Their intersection is empty, so a sound certificate cannot accept a single history-dependent action for both states.

C. Trusted Anchors and Learning-Aware FDI

A trusted state anchor can restore learning by shrinking the plausible-state set. At $\rho = 0.9$, a trusted sign anchor reduces the set to either $[0, 0.9]$ or $[-0.9, 0]$, yielding nonempty kernels $[-0.2, -0.08]$ and $[0.08, 0.2]$, respectively. This illustrates the escape condition: trusted information can make the kernel nonempty again.

Learning-aware FDI creates the complementary failure mode. If malicious update data proposes a candidate action $u = 0.1$, then the gate allows it at $\rho = 0.6$, projects it to 0.04 at $\rho = 0.8$, and rejects it at $\rho = 0.9$. The same candidate update is therefore safe, constrained, or forbidden depending on the certified kernel induced by the attacked history.

D. Lifecycle Benchmark Results

We next compare five mechanisms over five update steps: an ungated learner, a nominal action filter, a robust action filter that gates only the applied action, an always-freeze baseline, and LifecycleGate with projection. The one-dimensional benchmark uses the same system as above. The linear tank benchmark uses a level-deviation model

$$x_{t+1} = Ax_t + Bu_t, \quad A = \begin{bmatrix} 1.08 & 0.04 \\ 0.02 & 1.03 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0.1 \end{bmatrix}, \tag{32}$$

with $|u_t| \le 0.15$ and $x_t \in [-1, 1]^2$. The nonlinear tank benchmark uses square-root inter-tank and drain flows,

$$\begin{aligned} h_{1,t+1} &= h_{1,t} + \Delta t(q_0 + u_t - q_{12}(h_t) - d_1(h_{1,t})), \\ h_{2,t+1} &= h_{2,t} + \Delta t(q_{12}(h_t) - d_2(h_{2,t})), \end{aligned} \tag{33}$$

where q_{12} and d_i are square-root flow terms. Its interval observer returns a box around the nominal attacked history,

and the one-step kernel is computed over a grid in that box. In all benchmarks, learning-aware FDI pushes the learner toward a larger positive action parameter.

Figure 4 shows the separation between current-action filtering and update certification. In the one-dimensional benchmark at $\rho = 0.8$, the nominal action filter has unsafe certified rate 1.0, and the robust action filter has stale certified-policy rate 1.0 because the applied action is projected but the learner parameter still drifts outside $K(\rho)$. LifecycleGate has zero unsafe and stale certified operation while keeping one nontrivial update out of five. At $\rho = 0.9$, the kernel is empty; any continued update is uncertified learning, so LifecycleGate freezes.

The linear tank benchmark reproduces the same pattern beyond the one-dimensional frontier. At $\rho = 0.8$, nominal filtering has unsafe certified rate 0.8, robust action filtering has stale certified-policy rate 0.8, and LifecycleGate keeps both rates at zero while allowing two updates out of five. At $\rho = 0.85$, the robust action filter is stale in all five steps, while LifecycleGate permits one constrained update. At $\rho = 0.92$, the kernel is empty and LifecycleGate freezes.

The nonlinear tank benchmark preserves the same separation when the kernel is computed from sampled nonlinear dynamics. At $\rho = 0.2$, nominal filtering is unsafe in four of five steps, robust action filtering is stale in four of five steps, and LifecycleGate keeps both rates at zero while allowing two constrained updates. At $\rho = 0.45$, the interval-observer box admits no common safe action; robust action filtering rejects the applied action but still allows uncertified learning, while LifecycleGate freezes the update.

E. Parameter-Level Update Gate

The previous studies use action-level projection to expose the lifecycle boundary. We also instantiate the gate at the parameter level for a linear policy $u = kz + b$ on the one-dimensional system. For each future attacked observation z in a grid, the ambiguity interval $[z - \rho, z + \rho]$ induces a safe-action kernel. Requiring $kz+b$ to lie in every such kernel gives a set of linear halfspace constraints on (k, b). The parameter gate projects the poisoned candidate update onto this certified parameter set.

Figure 5 shows the resulting parameter-level frontier. At $\rho = 0.7$, current-action filtering has stale certified-policy rate 1.0 while the parameter gate keeps stale rate 0 and still permits two updates out of five. At $\rho = 0.8$, the parameter gate permits one constrained update. At $\rho = 0.85$, the current kernel is empty and the sound parameter-level mechanism freezes.

We then apply the same construction to the two-dimensional linear tank policy $u = k_1 z_1 + k_2 z_2 + b$. Each grid observation $z = (z_1, z_2)$ induces a plausible state box and a one-step safe-action interval, producing halfspaces in (k_1, k_2, b). Figure 6 shows that the separation persists beyond the scalar policy. At $\rho = 0.5$, current-action filtering has stale certified-policy rate 1.0, while the parameter gate has stale rate 0 and permits all five projected updates; the final projected parameters remain inside the future-history halfspace system.

F. Adaptive Attacker Stress Test

The fixed poison direction is not the full threat model. We therefore add a finite-search white-box attacker that chooses among poisoned update directions to maximize stale-policy, unsafe-action, or forced-freeze objectives. For forced-freeze attempts, the attacker also chooses the largest ambiguity radius inside its budget that most shrinks the certified set.

Figure 7 shows that the lifecycle separation is not an artifact of a fixed update direction. Under stale and unsafe objectives, the robust action filter preserves the applied action but leaves the raw policy stale in every step at high ambiguity. LifecycleGate keeps unsafe, stale, and uncertified-learning rates at zero, while still allowing projected updates when the certified set is nonempty. Under the forced-freeze objective, once the budget reaches $\rho = 0.84$, the common kernel is empty. The robust action filter rejects the applied action but still records five uncertified learner updates, whereas LifecycleGate rejects the certificate and performs zero updates.

G. Safe-Control-Gym Cartpole Semantics

We next instantiate the same lifecycle condition in Safe-Control-Gym cartpole as a standard-control benchmark smoke study. The baseline controller is the official LQR controller augmented with a two-parameter linear residual learner. The attacker injects continuous observation FDI and optimizes a CEM objective over stale-policy, unsafe-action, and forced-freeze margins. The attacked observation z and radius ρ induce the plausible box $X_{\mathcal{A}}(z) = \{x : \|x - z\|_\infty \le \rho\}$. For the main Safe-Control-Gym study, $K_\Gamma(z)$ is computed by sampling this box and evaluating the official Safe-Control-Gym CasADi discrete cartpole dynamics over an action grid. We also include a smaller reference run using the official cartpole CBF barrier and Lie-derivative condition as a sampled kernel.

This experiment is a direct semantic implementation of Theorem 1 and Proposition 3. The future-history abstraction $\mathcal{Z}$ is a local grid of future attacked observations around the current attacked observation. For each $z' \in \mathcal{Z}$, the robust kernel produces an interval $K_\Gamma(z') = [\ell_{z'}, u_{z'}]$. The LQR term is fixed, while the residual learner is linear in $(\theta_{\text{gain}}, \theta_{\text{bias}})$, so requiring the updated residual policy to stay inside all future kernels yields the halfspace constraints used by LifecycleGate.

Figure 8 separates three effects that are conflated by ordinary controller performance metrics. First, action filtering can remove current constraint violations without certifying the updated residual policy. Second, always-freeze avoids stale certification but has zero learning availability. Third, LifecycleGate projects the poisoned update into the future-history halfspace set when feasible, so stale and uncertified learning remain zero while updates continue before the freeze frontier. This run should be read as benchmark evidence that the lifecycle failure appears beyond the toy systems, not as a final high-fidelity industrial deployment. At larger ambiguity radii, the frontier panels show that the kernel becomes empty for action-only filtering; LifecycleGate then reduces or freezes learning instead of treating the previous certificate as valid for the updated policy.

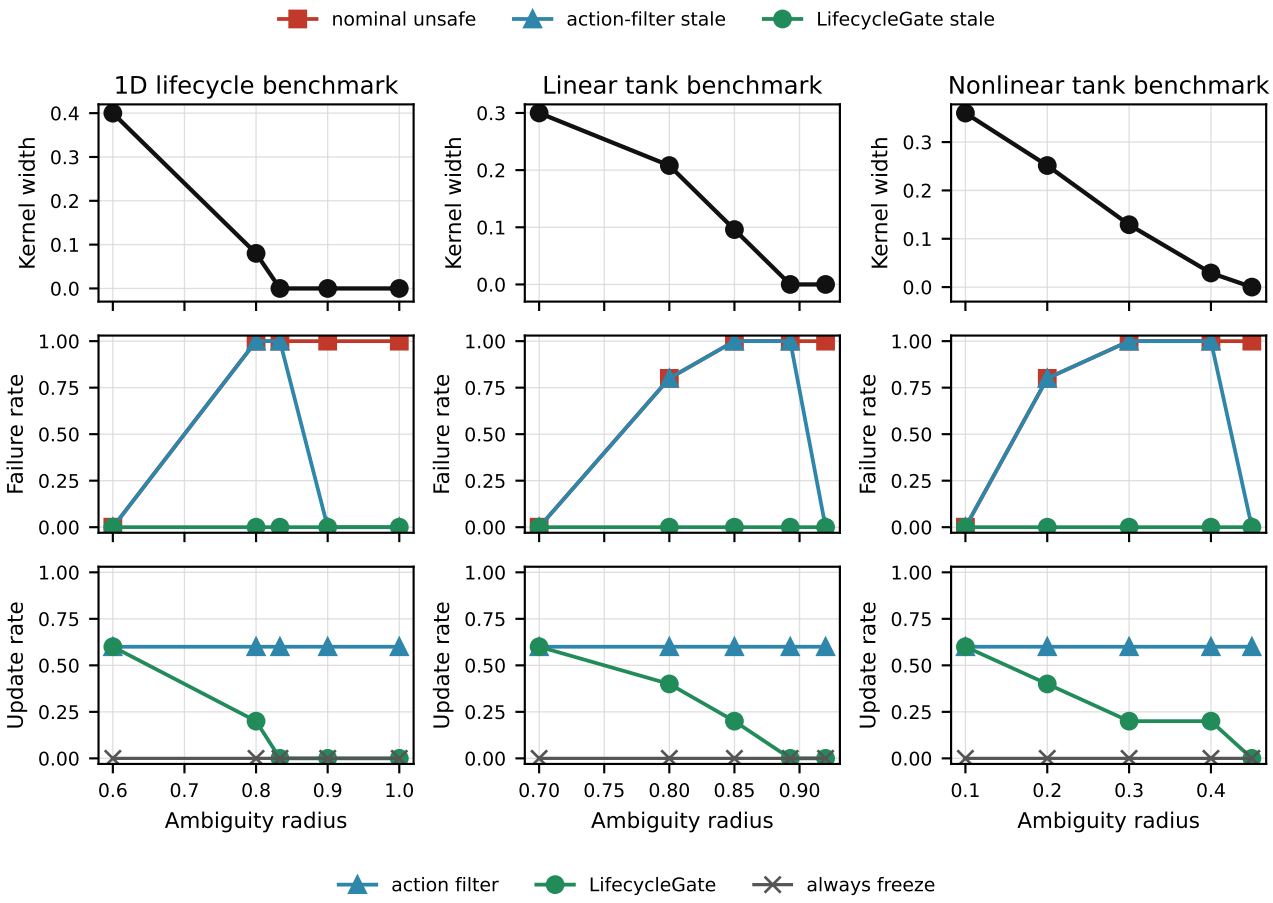

Fig. 4. Lifecycle-gate benchmark artifacts. As ambiguity grows, the certified kernel shrinks across one-dimensional, linear tank, and nonlinear tank models. Nominal filtering accepts unsafe certified actions, while robust action filtering can keep the applied action safe but leave the updated raw policy outside the kernel. LifecycleGate eliminates unsafe and stale certified operation by constraining or freezing updates, retaining some learning before the freeze frontier.

H. Prospective Contract-Separation Audit

All results in this subsection are conditional on the exposed reward-ingestion interface in Table I. This paper does not claim to bypass authenticated provenance or trusted reward recomputation. We first audit the paper's central contract transfer. Earlier development grids selected target $(18, -5)$ by five-step acceptance and 120-step harm. Before the sequential V3 audit, we locked that target, the bounded perturbation $2\tanh(\epsilon_t(\mu_t^\star - \mu_t))$, learner, state pools, and stop rule. Seed 2070 served as the development/canary run; after it passed the hard gate, seeds 2071–2072 were untouched confirmations and the protocol remained unchanged.

All 72 poisoned snapshot–state pairs pass the initial five-step check. Poisoned permanent raw release then fails on 11, 5, and 7 pairs per seed, versus paired clean counts 0, 0, and 2. Thus 21 of 23 poisoned failures are poison-only, two are shared, and none are clean-only (two-sided exact paired $p = 9.537 \times 10^{-7}$). The CasADi audit identifies the same 23 failures. Resident predictive authority incurs 0/72 failures and switches four to five steps before each corresponding poisoned raw-release failure.

We then froze the detector and opened five new learner/evaluation seed pairs once, without changing the target, attacker, learner, budget, state selection, or contracts. All 120 pairs pass the initial five-step check. Poisoned raw release fails on 5, 4, 7, 2, and 9 pairs across the five seeds, versus clean counts 2, 0, 0, 0, and 0. Resident predictive authority incurs 0/120 failures and switches before all 27 poisoned failures. The 25 poison-only versus zero clean-only discordances give two-sided exact $p = 5.960 \times 10^{-8}$; all five seeds reproduce poisoned failure, and all adaptation, budget, and pairing checks pass. As in the cross-system study below, state–snapshot pairs sharing a learner seed are not independent learner replicates, so the V3 and V4 exact tests summarize within-snapshot discordance. Cross-seed replication is conveyed instead by failure reproducing in all three and all five seeds, respectively.

For independent replication and the one-step negative control, older snapshots from seeds 2040–2042 use disjoint pools 8040–8042. Seventy-one of 72 states pass the initial check; all mechanisms share state–snapshot keys.

Table IV makes the three evidence blocks and deployment contracts explicit. A locked horizon sweep reuses the V3 snapshots and states without retraining. Horizons 3 and 5 each

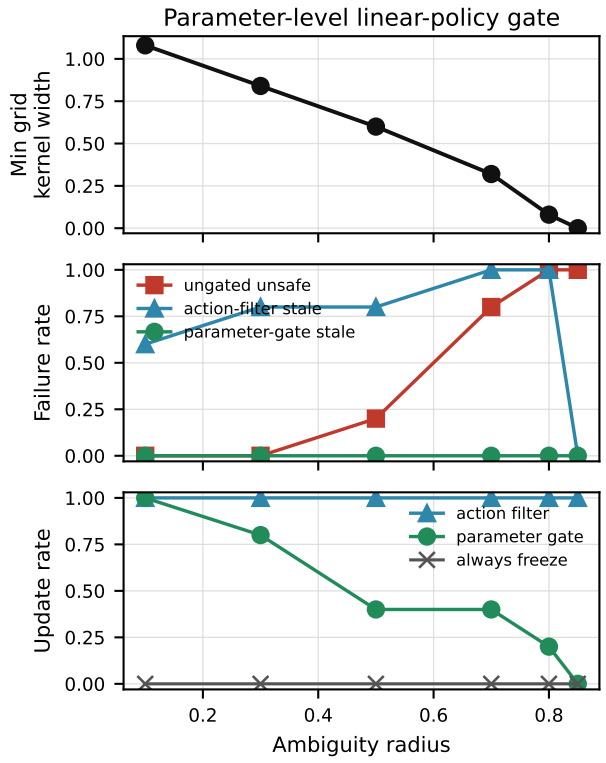

Fig. 5. Parameter-level update gating for a linear policy $u = kz+b$. Current-action filtering keeps the applied action safe but leaves the future action map stale. Projecting the parameters onto the certified halfspace set eliminates stale certified policies and reduces learning as the certified set shrinks.

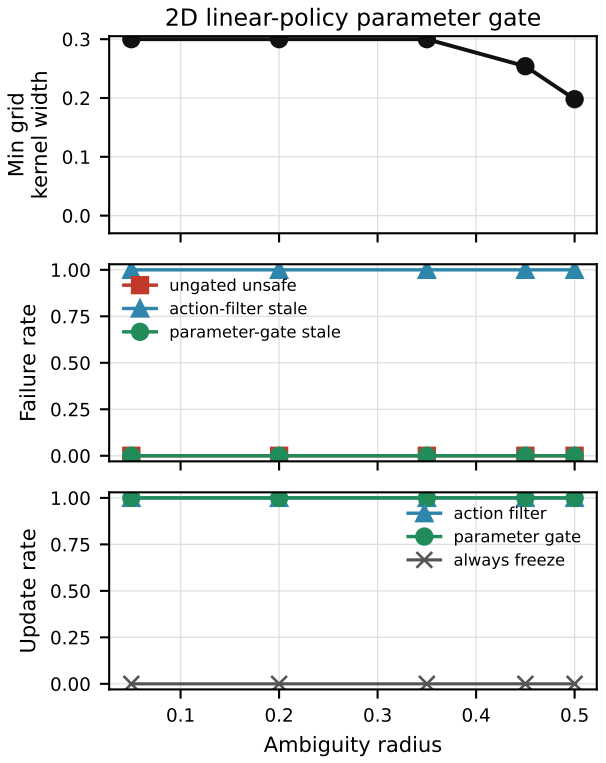

Fig. 6. Parameter-level update gating for a two-dimensional linear tank policy $u = k_1 z_1 + k_2 z_2 + b$. The certified parameter set is the intersection of future-history halfspaces over a grid of attacked observations. Current-action filtering leaves the future action map stale, while the parameter gate projects the poisoned update into the certified set.

retain 72 accepted pairs with 23 poisoned raw failures and zero resident failures; horizon 10 retains 57 accepted pairs, eight poisoned failures across two seeds, and zero resident failures. Thus three horizons meet the predeclared separation condition. Horizon 1 leaves 12 resident failures, exposing insufficient lookahead, while horizon 20 admits 49 pairs but no poisoned failures, exposing conservative rejection rather than a post-hoc replacement horizon.

In the independent audit, permanent raw release fails on 6/24, 3/23, and 10/24 accepted pairs. Resident predictive authority repeats the same five-step raw-policy prediction at every physical step and switches on exactly those 19 paired states, four steps before failure; no physical failure remains. In contrast, the one-step resident kernel fails on 14 pairs despite 79 interventions and reaches an empty kernel 41 times. We therefore claim neither that a finite-horizon pass proves invariance nor that residence alone is sufficient. The supported result is narrower: the same finite prediction is effective as a repeatedly evaluated recoverability trigger, but unsound as a one-time authorization for an unmonitored future.

I. Reward-Influence Geometry and Optimizer Boundary

We first validate the reward-influence analysis on a real, unclipped cartpole batch from learner seed 2040. Reconstructing the implemented gradient from Equation (15) has error 2.78×10^{-17}. The clean gradient norm is 0.196789, neither gradient nor parameter clipping is active, and the minimum centered-return norm over the reward box is 7.993864. Across 22 release-gate halfspaces, the largest discrepancy between an SOCP witness and its computed support is 1.07×10^{-9}. The minimum robust margin is 4.598004, so no halfspace can be crossed in this batch. This negative result matters: the successful physical attacker is the locked bounded $\tanh$ target-shaping rule and accumulates changing batch-local influences; it is not an SOCP-generated one-step gate crossing.

The homogenized clipped formulation is then audited on all 36 poisoned V3 batches. The old premises make 0/36 eligible; the new formulation yields verified support witnesses on 22/36, including 18 whose reward boxes contain a normalization singularity and 20 for which the old global gradient-clipping exclusion fails. Every actual update is below its computed support, all witnesses respect the reward budget, final parameter reconstruction error is zero, and maximum witness/support error is 3.67×10^{-5}. Ten batches cannot globally exclude coordinatewise parameter clipping (nine activate it on the observed edit), and four cone-optimal directions have no recovered witness above the learner's normalization tolerance;

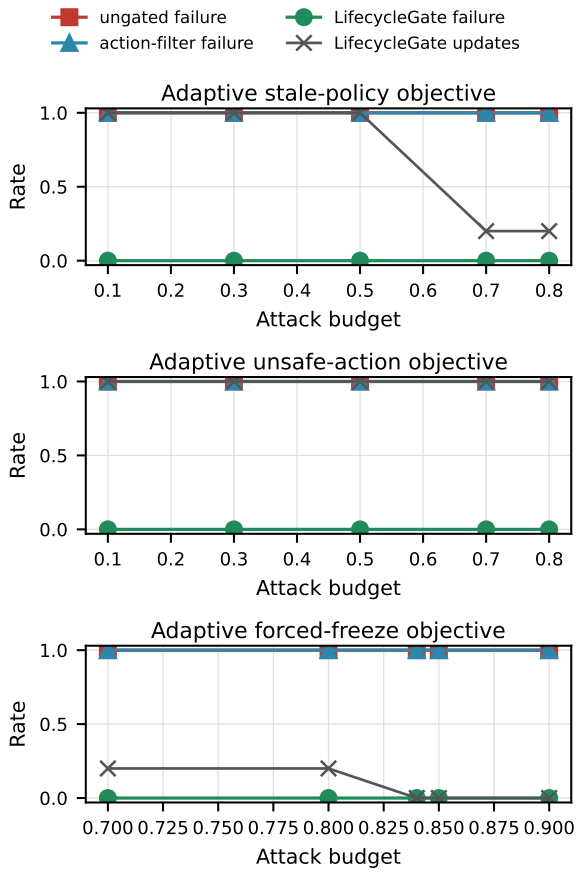

Fig. 7. Finite-search adaptive attacker against the scalar parameter gate. The attacker chooses update directions for stale-policy and unsafe-action objectives, and ambiguity radii for forced-freeze objectives. LifecycleGate keeps certified failure rate at zero; when the kernel is empty, it freezes instead of allowing uncertified learning.

they remain outside the exact implementation audit. The clipped quadrotor trajectories have not received this separate parameter-clipping audit and remain empirical.

Exact batch support still does not explain the successful multi-batch attack. The locked $\tanh$ update advances in the target direction on only 6/20 eligible batches with positive oracle increment, and its preregistered same-trajectory advantage reaches 22/36 rather than the required 27/36. The original locked exact-support smoke emitted 0/12 edits under the former singularity stop rule. A post-stop rerun on that already burned seed using the new solver emits nonzero edits in 9/12 batches, but its accepted raw snapshot still causes 0/24 release failures. We therefore use exact support as a batch-audit theorem, not a multi-batch attack optimizer; the physical result supports the fixed target and bounded attacker, not joint gate optimality of the $\tanh$ rule.

J. Cross-System Reward-Log Poisoning

We next test whether the lifecycle failure arises when updates are produced by an actual learner. The policy is a Gaussian residual actor on top of a trusted controller. Standard reward-to-go REINFORCE gradients are computed from PyBullet transitions. The attacker changes only the scalar reward written to the update log, subject to a per-step L_∞ budget; it cannot write the action, gradient, parameter, optimizer state, dynamics, or deployment state. A backup action filter protects the adaptation trajectory. Deployment then audits the raw learned snapshot, the object requiring lifecycle admission.

Training, certificate, calibration, and deployment states are separated by seed. All mechanisms within a learner seed use paired state sets. Candidate attack directions and hyperparameters are fixed on a development seed before the three-seed audits. The exact tests below operate at the paired deployment-state-block level, conditional on the three learned snapshots. Because blocks sharing a learner seed are not independent learner replicates, the p-values summarize within-snapshot discordance; replication across learners is conveyed separately by consistency over 3/3 seeds. Wilson and paired-block bootstrap intervals have the same conditional scope and are not confidence statements over a population of learner seeds.

On cartpole, the clean learner violates 1/188 admitted rollouts and the poisoned action-only learner violates 38/188. There are 37 paired poison-only failures and no clean-only failures (two-sided exact paired $p = 1.455 \times 10^{-11}$). Both commit admission and always-freeze have zero observed violations.

On two-dimensional quadrotor, clean learning has 0/72 violations, while reward-only poisoning causes 68/72, all delayed, across 3/3 learner seeds. The paired discordance is 68 poison-only to zero clean-only (two-sided exact $p = 6.776 \times 10^{-21}$). Commit admission and always-freeze again have zero observed violations. The gate retains nonzero fractions 0.55, 0.40, and 0.70 of the three poisoned pending updates. These fractions establish update availability under attack, not benign

TABLE IV
CONTRACT SEPARATION AFTER INITIAL FIVE-STEP ACCEPTANCE, CONDITIONAL ON BOUNDED REWARD-RECORD WRITE ACCESS. V4 IS THE UNTOUCHED FIVE-SEED CONFIRMATION; V3 RECORDS DEVELOPMENT AND THE EARLIER SEQUENTIAL AUDIT. THE INDEPENDENT AUDIT RETAINS THE ONE-STEP NEGATIVE CONTROL. A SWITCH PERMANENTLY HANDS CONTROL TO TRUSTED LQR.

Contract	Failures	Authority event
Untouched V4 confirmation (120 pairs; 5 seeds)		
Clean permanent raw release	2/120	—
Poisoned permanent raw release	27/120	—
Resident predictive authority	0/120	27 switches
Development/sequential V3 (72 paired states)		
Clean permanent raw release	2/72	—
Poisoned permanent raw release	23/72	—
Resident predictive authority	0/72	23 switches
Independent audit (71 accepted states)		
Permanent raw release	19/71	—
One-step resident kernel	14/71	79 int.; 41 empty
Resident predictive authority	0/71	19 switches

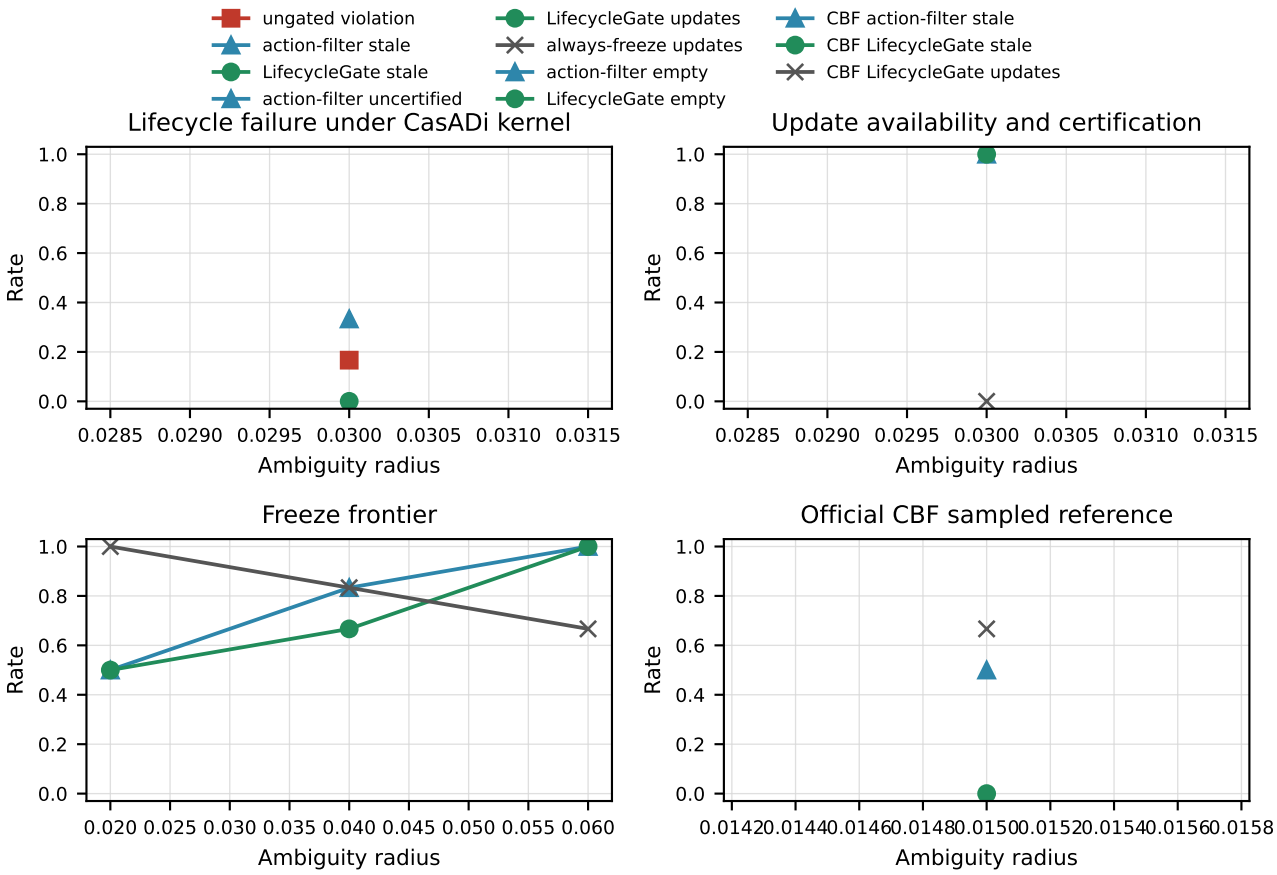

Fig. 8. Safe-Control-Gym cartpole lifecycle evidence from a short two-seed sweep. The main panels use the CasADi discrete-dynamics robust kernel with an Euler surrogate for adaptive attack search. At $\rho = 0.03$, the ungated learner has constraint-violation rate 0.167, and the action-only plausible filter removes current violations but leaves stale certified policies at rate 0.333 with uncertified-learning rate 1.0. LifecycleGate keeps stale and uncertified-learning rates at zero while retaining update rate 1.0. The freeze-frontier sweep shows the robust kernel emptying as ambiguity grows, and the CBF sampled reference reproduces the same stale-update separation with an official barrier kernel.

learning utility.

A learned value baseline does not blunt the attack. Under a protocol specified and locked before execution, we swap REINFORCE's constant batch-mean baseline for a learned per-state linear value baseline. This advantage variant shares the same Monte-Carlo return, residual actor, score function, and reward-log channel; the trusted learner fits the baseline from logged rewards. Across three untouched seeds, poisoning causes 40/126 raw-release failures versus 2/126 paired clean, with commit and always-freeze at 0/126 and all three seeds failing (38 poison-only versus zero clean-only, two-sided exact $p = 7.3 \times 10^{-12}$; within-snapshot scope, generality carried by 3/3 seeds). This is a robustness check within the Monte-Carlo policy-gradient family, not a distinct learner family: off-policy or bootstrapped learners, which would change the attack channel through a temporal-difference target, and the negative official-PPO result remain the learner-generality boundary. Absolute rates use the full 42-state envelope ($126 = 3 \times 42$) without the initial-acceptance filter, so they are not directly comparable to the filtered 120-pair hero cohort.

Non-singleton adaptation set. The general history model (Section III) also permits observation FDI. As a *strictly stronger* attacker than the reward-log-only core, we additionally bias the pole-angle observation by a radius ρ, so the adaptation-time protective kernel ranges over a plausible box of half-width ρ (about $\pm 2.3^\circ$ at $\rho = 0.04$); the release certificate and deployment states are unchanged. Under a locked ρ grid over three untouched seeds the contract separation persists at every non-singleton radius: poisoned raw release fails on 44/126 and 61/126 pairs at $\rho = 0.02$ and 0.04 versus 4/126 and 3/126 clean, while commit and always-freeze remain 0/126 (the near-singleton reference $\rho = 0.005$ is 41/126 versus 5/126). The per-seed raw-failure count is not monotone in ρ, so we claim persistence under a non-singleton adaptation set, not a monotone trend; the kernel does not empty on this grid, so this is separation persistence, not the analytic freeze frontier. The end-to-end result is thus not an artifact of a singleton plausible state.

TABLE V

PAIRED REWARD-LOG POISONING AND SNAPSHOT ADMISSION. "COMMIT" DENOTES FINITE-HORIZON ADMISSION; PERMANENT MECHANISMS USE THE COMMON-STATE AUDIT IN FIG. 9. RAW-SNAPSHOT REWARDS AFTER FAILURE ARE DESCRIPTIVE, NOT UTILITY COMPARISONS.

System	Mechanism	Learner seeds	Rollouts	Violations	Mean reward
Cartpole	Clean REINFORCE	3	188	1	-0.197
Cartpole	Poisoned action-only	3	188	38	-1.269
Cartpole	Poisoned commit	3	188	0	-0.102
Cartpole	Always-freeze	3	188	0	-0.117
Quadrotor	Clean REINFORCE	3	72	0	-0.030
Quadrotor	Poisoned action-only	3	72	68	-0.170
Quadrotor	Poisoned commit	3	72	0	-0.060
Quadrotor	Always-freeze	3	72	0	-0.029

TABLE VI

FROZEN REWARD-INTEGRITY AUDIT. RATES ARE BATCH-LEVEL TPR/FPR ON SEVEN EVALUATION SEEDS; EACH PREREQUISITE IS AN ADDITIONAL DEFENDER ASSUMPTION.

Check	Required trusted information	TPR/FPR
Reward recomputation	Authenticated transition and deterministic reward	1.000/0
Known-sign check	Task guarantee that reward is nonpositive	0.905/0
Scalar envelope	Clean development range transfers	0.917/0.036
Batch-mean envelope	Clean batch mean transfers	0.012/0.036

K. Reward-Log Integrity and Detectability Boundary

The attack demonstrates a release-contract failure, not stealth against a reward-integrity monitor. We freeze four checks on clean development seed 2070, then evaluate 84 clean and 84 poisoned batches from seeds 2071–2072 and the five untouched V4 seeds.

Table VI shows that trusted recomputation blocks every evaluated edit, and even task-specific scalar checks detect over 90% of poisoned batches. The ineffective batch-mean envelope shows why a generic "monitor the average" statement is not enough: detection depends on reward semantics and trusted inputs. Cryptographic origin binding would also reject post-producer mutation if the trusted producer signs the reward together with its transition, action, and time; this experiment does not evaluate compromise of such a trusted producer.

We also test whether temporal or moment-preserving edits recover both efficacy and lower detectability on the already burned development seed. Sparse permutation and projected moment-preserving variants both satisfy their integrity constraints and move parameters toward the harmful target, but each causes 0/24 raw-release failures. The work package stops without opening a new seed namespace. Thus the supported attacker is effective but readily detectable by simple task-aware checks; authentication or trusted reward recomputation is a valid upstream escape condition. Commit admission and resident predictive authority remain downstream containment mechanisms, not substitutes for log integrity.

In-loop restatement of the check. Placing the same known-sign check *inside* the update loop as a batch gate that freezes any batch whose logged reward violates the nonpositive guarantee (under a locked protocol) reduces poisoned raw-release failures from 39/126 to 0/126 across three untouched seeds (35 of 36 poisoned batches frozen; paired defended-versus-undefended two-sided exact $p = 3.6 \times 10^{-12}$; within-snapshot scope, 3/3 seeds). Its safety only matches the always-freeze baseline; the added value is *selectivity*: it freezes zero clean batches, whereas under attack it freezes nearly every batch, so availability collapses to freeze exactly when the attack is active. It catches only the sign-violating effective attack; the sign-respecting moment-preserving variants above evade it yet independently cause no deployment harm here. It is therefore an in-loop restatement of the detectability boundary rather than a defense against a sign-valid adaptive attacker, and complements rather than replaces upstream provenance or recomputation.

L. Runtime Escape Conditions and Their Cost

Snapshot admission is not the only sound architectural response. If a runtime filter is permanently present and its certificate covers the composed policy $F \circ \pi$, raw-policy failure need not become physical failure. A prior audit established feasibility, but used only 12 rollout blocks rather than the 72 blocks used for the raw and commit mechanisms. We therefore lock a new result-independent pool of 144 states from three disjoint seeds. For each learner seed, the common operating set contains exactly the states certified for both freeze and that seed's committed snapshot at horizon 100 and guard 0.003. We report its coverage, select eight order-spanning states, and execute all six mechanisms from identical state/reset pairs. Thus every pooled mechanism has 24 paired rollouts and 2400 possible steps.

Table IX and Fig. 9 show the resulting separation. The poisoned raw snapshots violate 23/24 rollouts and complete only one. Clean, commit, freeze, the permanent five-step backup shield, and Safe-Control-Gym's official linear MPSC all have zero observed failures and complete 24/24. The shield intervenes on 511/2400 steps with no rejection, while MPSC intervenes on 2400/2400 with no infeasible solve. Median/p95 online latency is 221.925/238.216 ms for the shield and 59.657/76.055 ms for MPSC, compared with 0.026–0.027/0.028–0.029 ms for raw snapshot evaluation. These

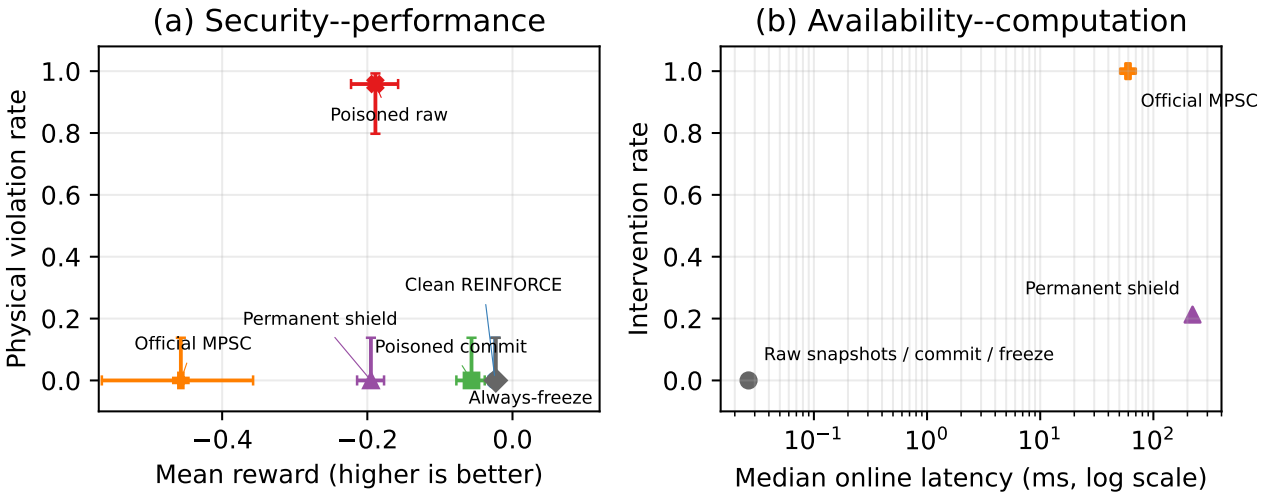

Fig. 9. The unified frontier on the same paired state blocks. Error bars in (a) are 95% Wilson intervals for violation rate and paired-block bootstrap intervals for mean rollout reward, conditional on the three locked learner snapshots. Panel (b) separates the negligible raw policy latency from the permanently-online protection cost; raw, commit, and freeze mechanisms overlap near 0.026 ms and zero intervention.

TABLE VII
COMMIT AVAILABILITY AND OFFLINE COMPUTATION ON THE INDEPENDENT 144-STATE CANDIDATE POOL. COMMON ADMISSION REQUIRES BOTH FREEZE AND THE COMMITTED SNAPSHOT TO PASS THE 100-STEP, 0.003-GUARD CERTIFICATE.

Seed	Common/144	Commit frac.	Offline (s)
2040	143/144 (99.3%)	0.55	8.49
2041	144/144 (100.0%)	0.40	9.32
2042	130/144 (90.3%)	0.70	8.50

single-process wall-clock measurements are not worst-case execution-time guarantees.

Safety does not make these mechanisms utility-equivalent. Mean reward is -0.023 for freeze, -0.024 for clean, -0.056 for commit, -0.195 for the shield, and -0.457 for MPSC. Paired rollout deltas relative to freeze are respectively -0.00096 (paired-block bootstrap 95% interval $[-0.00141, -0.00055]$), -0.03350 ($[-0.04754, -0.02182]$), -0.17208 ($[-0.18601, -0.15886]$), and -0.43393 ($[-0.54016, -0.33763]$); none improves on freeze in any of 24 blocks. This is a performance frontier under the locked attack evaluation, not a benign adaptation-utility experiment.

Table VII makes the snapshot-admission cost explicit. The common operating-set coverage is 99.3%, 100.0%, and 90.3% for learner seeds 2040–2042. Reconstructing trusted-state admission and the 21-point contiguous backtracking takes 8.49–9.32 s per seed and exactly reproduces the locked commit fractions 0.55, 0.40, and 0.70. Offline commit therefore avoids permanent per-action computation but pays finite certificate coverage and update latency. The comparison distinguishes three contracts: certify a snapshot before raw deployment, retain a filter permanently, or stop adaptation.

M. Trusted-State-Anchor Infrastructure

A trusted state anchor shrinks certificate uncertainty but does not sanitize the reward record. The artifact varies anchor period $P \in \{1, 3, 6, 12\}$, requiring 12, 4, 2, or 1 calls over 12 updates. Every condition retains all 16 certificate centers and has 0/24 violations on new paired blocks. Periods 1–6 preserve retained fractions 0.55/0.40/0.70; at $P = 12$ they become 0.50/0.40/0.65. Neither quality grid changes fractions at 0.05 resolution, and no condition improves reward over freeze. Full uncertainty, timing, and paired-reward rows remain in the artifact; hardware latency, energy, authentication, and monetary cost are not measured.

N. Boundary I: Official PPO

The official pretrained PPO policy is safe on 72/72 held-out rollouts, while a separately constructed malicious target is unsafe on 72/72. Under the locked $L_\infty \leq 0.5$ reward budget, the 16-batch checkpoint has 0/24 clean and 1/24 poisoned failures; at 24 batches both have 0/24. The pre-specified physical and two-checkpoint margin conditions fail, so no multi-seed sweep is run. This supports update redirection, not a stable deep-PPO unsafe-policy attack.

O. Boundary II: Benign Adaptation Utility

In the locked benign-utility smoke under a 0.003 N differential actuator bias, freeze, clean adaptation, and LifecycleGate all have 0/12 violations and mean rewards -0.03617, -0.04714, and -0.08097. LifecycleGate accepts a nonzero update in 11/12 batches (mean fraction 0.8208) but improves no paired rollout over freeze. The stop rule fires, so no formal three-seed experiment is run: parameter motion and certificate acceptance do not establish useful adaptation.

P. Measurement and Validity Boundaries

The CasADi gates use fixed finite state sets; physical outcomes use independent PyBullet rollouts. They are sampled finite-horizon certificates, not continuous-state invariant-set proofs. For the benign shift, maximum action implementation error is 2.61×10^{-11} N. A corrected calibration metric leaves seeds, states, bias grid, and threshold unchanged: realized state violations and pre-clip saturation are reported separately.

IX. Related Work and Discussion

Sensor-attack safety and secure estimation. Secure estimation characterizes when attacks are detectable or correctable [12], [19]; severe-attack CBF filtering constructs plausible-state sets and safe-action kernels when reconstruction is not unique [10]. We start after this boundary: given the state set a filter can justify, must an adversarially updated controller remain inside its certificate?

Runtime assurance and online control upgrades. Simplex retains a decision module and baseline during online upgrades [1]; Neural and barrier-based Simplex add online retraining and forward/reverse switching [2], [3], but protect a resident composition. We instead pair the same predictor before and after the contract transition that removes its authority. Model-free adaptive predictive control under deception or denial-of-service bounds tracking or frequency recovery [20], [21] certifies no safety envelope for an adversarially updated policy at release.

Safety filters and shielded learning. Linear/adaptive MPSC and shields preserve recoverability or forbid unsafe actions [22]–[24]. Shielded transfer uses separate generalization evidence before removal [11]. We distinguish two roles that the prior literature has not separated. Hsu et al. [11] provide a correct negative control: authority transition preceded by independent transfer evidence. Carr et al. [9] define the opposite design, a shield bootstrap-to-retirement workflow under partial observability that compares sudden and smooth shield removal; their learning signals are trusted, and they do not consider an adversarial update log. We use that published lifecycle as a third-party case study in Section IV and add the update-data integrity dimension: a bounded reward-record adversary can invalidate a short release predictor that remains effective while resident. Proposition 1 states the formal obligation that separates these roles: each evidence-tuple component changed by the transition must be re-established under the deployment contract. A permanent filter certifies $F \circ \pi$, while raw release requires evidence for π; our question is whether a bounded reward-record adversary can invalidate a short release predictor that remains effective while resident. Adjacent lines learn or certify safety online through barrier learning [25], bandit safe control [26], certified reinforcement-learning robustness [27], [28], and adversarial reach–avoid safety filters [29], [30]; each assumes trusted learning signals rather than an adversarial update log.

Policy and certificate updates. Policy repair, safe projection, joint controller/certificate repair, and robust conformal transfer certify or preserve behavior across updates [4]–[6], [31], [32]. Our narrower contribution is an attack-specific reward-to-gate analysis and paired evidence against transferring a resident reverse-switch check into raw release.

Policy and reward poisoning. Policy poisoning is known for batch reinforcement learning and control [33]; online attacks perturb each reward under an ℓ_∞ budget and adapt to the learning process [34], including policy-gradient learners without known dynamics [35] and black-box deep-RL algorithms [36]. Targeted reward or environment poisoning can teach a chosen policy [37], and a corrupted reward channel is a known reinforcement-learning hazard [38]. Closest in the control setting, backdoor attacks on low-level controllers poison operational data of PID/LQR loops to implant a latent vulnerability activated by an exogenous physical signal [39]. Reward poisoning itself is therefore not our novelty. Unlike a trigger-conditioned controller backdoor, our deployed failure is unconditional after raw release and is defined against a runtime-assurance certificate rather than an activation input. We connect this attack channel to certificate inheritance in a physical controller: adaptation actions can remain protected while the raw deployable snapshot fails later, and snapshot admission, freeze, and permanent filtering expose different physical, coverage, and availability outcomes. Online reward-poisoning defenses also model adaptive attacker–defender interaction [40]; our integrity audit is more architectural, distinguishing authenticated provenance, trusted recomputation, semantics-dependent detection, and downstream runtime containment rather than proposing a new poisoning detector.

Fractional and conic optimization. Fractional programs can be homogenized into equivalent convex forms under appropriate numerator and denominator structure [14]; disciplined quasiconvex programming systematizes the resulting fixed-level feasibility and bisection pattern [17]. Projection and feasibility over homogenization and second-order cones are classical objects [15], [16]. We claim no new conic tool: we instantiate them for the normalized image of a centered reward-to-go zonotope and show that global gradient-norm clipping introduces a second SOC denominator branch while unit-positive-numerator normalization removes the singular feasible point.

Discussion. A permanent sound filter can protect the composition $F \circ \pi$, and our resident predictive authority and MPSC results confirm this escape condition; it does not certify a raw updated snapshot after the filter is removed. The third-party case study in Section IV shows the same authority-transition boundary on a published shield-retirement lifecycle, so the contract distinction is not an artifact of our benchmark. Conversely, the failed one-step resident kernel shows that residence alone does not suffice: the decision rule must preserve recoverability. Snapshot admission, resident predictive authority, action filtering, always-freeze, and trusted re-anchoring are different assurance contracts.

The evidence has eight important limitations. First, the necessary gate and conditional finite-abstraction lift are not complete controller-synthesis results: the lift assumes a justified joint reachable-set cover and known regularity/model-

error bounds. Second, the parameter and commit gates use finite future-history or state samples without such a cover proof; their soundness is limited by abstraction, model mismatch, horizon, and guard margin. Third, the five-step contract horizon and attack target were developed on earlier seeds; the five-seed untouched confirmation and locked horizon sweep reduce but do not erase that history. Fourth, the bounded $\tanh$ reward rule is target-conditioned rather than a joint gate optimizer; the extended exact-support diagnostic emits nine nonzero edits but still causes no raw-release failure. Fifth, the exact reward-influence theorem covers global gradient clipping but not coordinatewise parameter clipping; 22/36 V3 poisoned batches pass the resulting implementation audit, and the clipped quadrotor attack remains empirical. Sixth, the effective attack begins after compromise of a bounded reward-ingestion principal; we neither demonstrate that service exploit nor claim universality across OT architectures. It is not stealthy to frozen task-aware reward checks and assumes absent origin integrity and trusted recomputation; the tested moment-preserving alternatives do not retain deployment harm. Seventh, the end-to-end physical attack is established for low-dimensional Monte-Carlo policy-gradient learners (REINFORCE and a learned-baseline variant sharing the same attack channel), not a genuinely distinct family; the official-PPO audit does not show a stable physical failure, and the benign-shift smoke shows no utility over always-freeze. On the third-party lifecycle the REINFORCE retirement-boundary isolation reproduces on 3/3 seeds, whereas the third-party PPO full-phase result is positive on 1/3 seeds; PPO retirement-boundary isolation remains the open learner-generality boundary. Eighth, the anchor audit models bounded error on only altitude and pitch and reports calls rather than platform-specific sensing latency, energy, or authentication cost. These are result boundaries, not omitted positive evidence.

X. Conclusion

Conditional on bounded write access at an otherwise isolated reward-ingestion interface, protected adaptation and raw release are different security contracts. The distinction is not an artifact of a self-defined benchmark. On the published shield-retirement lifecycle of Carr et al., bounded reward-log poisoning confined to the shielded bootstrap phase already makes the raw policy 1.6–$3.6\times$ more dangerous at retirement on 3/3 paired seeds, and the damage persists after fully clean post-retirement learning on 2/3. A controlled Safe-Control-Gym study then isolates the mechanism: after a development block locks the target, attacker, and detector, five untouched seeds yield 27/120 poisoned permanent-release failures versus 2/120 paired clean. Resident predictive authority switches before every corresponding poisoned failure and incurs 0/120 failures; horizons 3, 5, and 10 all separate the contracts. An independent audit finds that a one-step resident kernel still fails on 14/71, so the result depends on recoverability, not residence alone.

The normalized reward-influence analysis characterizes the batch-local update authority available through reward corruption. Conic homogenization makes positive support exact across normalization singularities and global gradient-norm clipping, while coordinatewise parameter clipping and multi-batch attack construction remain outside the theorem. The broader cartpole and quadrotor experiments, finite-horizon commit gate, freeze, backup shield, and official MPSC then expose the coverage, intervention, reward, and latency costs of different escape contracts.

The integrity boundary is also part of the result. Trusted recomputation blocks every evaluated edit, and simple task-aware checks detect over 90% of poisoned batches. Provenance or recomputation removes the upstream attack precondition; commit admission and resident predictive authority contain its downstream consequence. We do not claim a logging-service exploit or a stealthy attack.

The remaining negative boundaries are part of the result: the target was development-fixed, the bounded $\tanh$ perturbation is not a joint gate optimizer, and even the extended exact-support diagnostic produces no raw release failure. The physical attack is established for low-dimensional Monte-Carlo policy-gradient learners (REINFORCE and a learned-baseline variant), does not stably transfer to official PPO, and admitted updates do not beat freeze in the benign-adaptation smoke. On the third-party lifecycle the REINFORCE retirement-boundary isolation reproduces on 3/3 seeds, whereas the third-party PPO full-phase result is positive on 1/3 seeds, leaving PPO retirement-boundary isolation as the open learner-generality boundary. Formally, an evidence tuple (s, a, p, i, τ) collected under the training contract must re-establish each component the deployment contract changes (Proposition 1). Thus resident predictive evidence is contract-bound; raw release needs independent future coverage.

Appendix A
Certificate Coverage and Model Mismatch

Reporting zero violations only on certificate-admitted states can be vacuous. We therefore lock a separate CasADi–PyBullet confusion audit before execution. It reconstructs the 12 clean, poisoned, commit, and freeze snapshots from the original traces, draws 144 states from three disjoint source seeds, performs no certificate admission, and compares horizons 20/50/100 and guard margins 0/0.001/0.003/0.005/0.01. The primary configuration is the pre-existing 100-step, 0.003-guard commit configuration; the grid is characterization rather than a selector.

Table VIII evaluates 1728 snapshot–state pairs. CasADi certifies 1293; none violates the PyBullet state box within 100 steps. Clean, commit, and freeze coverage is 0.986, 0.944, and 1.000. The raw poisoned snapshots physically violate 403/432 pairs and have certificate coverage 0.062, so their low coverage is informative rejection rather than vacuous evidence for a safe deployed snapshot. Across all mechanisms the certificate conservatively rejects five physically safe pairs. Maximum action-interface error and actuator saturation are zero.

The grid exposes a model-mismatch boundary. Without a guard, poisoned snapshots produce 1, 2, and 1 false accep-

TABLE VIII
CASADI–PYBULLET COVERAGE AT THE PRE-EXISTING 100-STEP, 0.003-GUARD CONFIGURATION. EACH ROW POOLS THREE LEARNER SNAPSHOTS OVER THE SAME 144 STATES.

Snapshot	Pairs	Physical viol.	Certified	False accept	False reject	Coverage
Clean REINFORCE	432	6	426	0	0	0.986
Poisoned action-only	432	403	27	0	2	0.062
Poisoned commit	432	21	408	0	3	0.944
Always-freeze	432	0	432	0	0	1.000

tances at horizons 20, 50, and 100. Every locked positive guard removes observed false acceptance, with increasing false rejection as the guard grows. Because 0.003 was declared as the primary pre-existing configuration before this audit, this is not post-hoc guard selection. Zero observed false acceptance on this finite distribution still does not establish continuous-state invariance.

APPENDIX B
UNIFIED PAIRED SECURITY–AVAILABILITY FRONTIER

APPENDIX C
CLOSEST-WORK COMPARISON

ETHICS CONSIDERATIONS

This paper studies an attack on safety-critical learning-enabled control, so we state its scope and safeguards explicitly. All experiments run in simulation (Safe-Control-Gym with PyBullet dynamics); no physical plant, vehicle, or deployed industrial system was involved, and no human subjects or personal data were used. The reported "failures" are simulated constraint violations used to compare deployment contracts, not incidents on any real system.

The evaluated threat begins *after* a reward-ingestion principal in the asynchronous learning-data plane has been compromised, and grants the attacker only bounded scalar writes to reward records. We deliberately do not demonstrate, and do not provide, any capability to obtain that foothold: we present no exploit against a historian, broker, replay service, or reward producer, and no technique to evade log-integrity controls. The contribution is thus not a new intrusion capability but a characterization of what a *known* least-privilege data-integrity gap implies for certificate inheritance at raw-policy release.

We judge the disclosure to be net beneficial. The same study that exposes the assurance-contract gap also identifies concrete, deployable countermeasures and measures their cost: origin-bound append-only reward records and trusted recomputation remove the upstream precondition entirely (recomputation blocks every evaluated edit), while resident predictive authority, commit admission, and permanent runtime filtering contain the downstream physical consequence. The effective attack is also readily detectable by simple task-aware reward checks. Surfacing this contract distinction, together with its defenses, before learning-enabled controllers are widely promoted from protected adaptation to unmonitored release reduces rather than increases real-world risk. We do not claim a fielded vendor vulnerability. Shield-retirement itself is not hypothetical: Carr et al. independently proposed and evaluated sudden and smooth shield removal as part of a published learning-to-deployment workflow, and we use that published lifecycle as a third-party case study rather than claiming a vulnerability in their system. We are not aware of a specific fielded industrial system exhibiting this exact pipeline, so no vendor disclosure was applicable; operators adopting protected-adaptation-then-release workflows are the intended audience for the mitigations.

OPEN SCIENCE

We support the NDSS open-science policy. The artifact includes row-level result CSVs, the exact commands and seed namespaces to reproduce every table and figure, upstream revisions and model files, dependency versions, source-linked generated tables, and SHA-256 checksums for all locked files, together with an automated audit that checks table provenance, claim locks, and format. We intend to release the code and data required to reproduce the paper's results under an open license upon publication. It retains the full anchor, coverage, timing, PPO, and utility boundaries; none authorizes post-hoc protocol changes.

REFERENCES

[1] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The simplex architecture for safe on-line control system upgrades," in *Proceedings of the 1998 American Control Conference*, vol. 6, 1998, pp. 3504–3508.

[2] D. T. Phan, R. Grosu, N. Jansen, N. Paoletti, S. A. Smolka, and S. D. Stoller, "Neural simplex architecture," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 12229. Springer, 2020, pp. 97–114.

[3] A. Damare, S. Roy, R. Sharma, K. DSouza, S. A. Smolka, and S. D. Stoller, "A barrier certificate-based simplex architecture for systems with approximate and hybrid dynamics," *IEEE Access*, vol. 13, pp. 157 742–157 763, 2025.

[4] W. Zhou, R. Gao, B. Kim, E. Kang, and W. Li, "Runtime-safety-guided policy repair," in *Runtime Verification*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 131–150.

[5] P. Lu, M. Cleaveland, O. Sokolsky, I. Lee, and I. Ruchkin, "Repairing learning-enabled controllers while preserving what works," in *2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems*. IEEE, 2024, pp. 1–11.

[6] Y. Chow, O. Nachum, A. Faust, E. Dueñez-Guzman, and M. Ghavamzadeh, "Safe policy learning for continuous control," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 801–821. [Online]. Available: https://proceedings.mlr.press/v155/chow21a.html

[7] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule, and M. Thompson, "Guide to operational technology (OT) security," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-82 Revision 3, 2023.

[8] MITRE ATT&CK, "Data historian, asset a0006," https://attack.mitre.org/assets/A0006/, 2023, accessed 2026-08-05.

TABLE IX

UNIFIED PAIRED SECURITY–AVAILABILITY FRONTIER. WILSON 95% INTERVALS ARE $[0, 0.138]$ FOR EVERY 0/24 VIOLATION COUNT AND $[0.798, 0.993]$ FOR POISONED RAW. INTERVENTION IS A FRACTION OF EXECUTED STEPS. † DENOTES REWARD TRUNCATED AT THE FIRST SAFETY FAILURE AND THEREFORE NOT A UTILITY COMPARISON.

Mechanism	Viol./rollouts	Complete	Mean reward	Interv.	Latency med./p95 (ms)
Clean REINFORCE	0/24	24/24	-0.024	0.0%	0.027/0.029
Poisoned raw	23/24	1/24	$-0.189^{\dagger}$	0.0%	0.027/0.029
Poisoned commit	0/24	24/24	-0.056	0.0%	0.026/0.029
Always-freeze	0/24	24/24	-0.023	0.0%	0.026/0.028
Permanent shield	0/24	24/24	-0.195	21.3%	221.925/238.216
Official MPSC	0/24	24/24	-0.457	100.0%	59.657/76.055

TABLE X

CLOSEST WORK ALONG THE THREE AXES THIS PAPER COMBINES: AN ADVERSARIAL *online update-data* CHANNEL, A RAW POLICY THAT *leaves* THE RUNTIME-ASSURANCE AUTHORITY WHOSE EVIDENCE CERTIFIED IT, AND INDEPENDENT EVIDENCE FOR THE RAW POLICY.

Work	Adversarial update log	Authority removed	Independent raw-policy evidence
Simplex / Neural Simplex [1], [2]	no	no (monitor resident)	n/a (composition certified)
Bb-Simplex [3]	no (trusted retraining)	no	n/a
Policy repair / MPSC [4], [22]	no (trusted evidence)	repaired, verified	repaired
Hsu et al. [11]	no	yes	yes (independent transfer evidence)
Carr et al. [9]	no	yes (sudden/smooth retirement)	empirical retirement; no formal claim
BADControl [39]	operational data, physical trigger	n/a (no RTA contract)	n/a
This paper, controlled study	**yes (reward log)**	**yes (permanent release)**	**attack tests transition evidence**
This paper, third-party case study	**yes (reward log)**	**yes (Carr retirement)**	**attack tests transition evidence**

[9] S. Carr, N. Jansen, S. Junges, and U. Topcu, "Safe reinforcement learning via shielding under partial observability," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, 2023, pp. 14 748–14 756.

[10] X. Tan, P. Ong, P. Tabuada, and A. D. Ames, "Safety of linear systems under severe sensor attacks," 2024. [Online]. Available: https://arxiv.org/abs/2409.08413

[11] K.-C. Hsu, A. Z. Ren, D. P. Nguyen, A. Majumdar, and J. F. Fisac, "Sim-to-lab-to-real: Safe reinforcement learning with shielding and generalization guarantees," *Artificial Intelligence*, p. 103811, 2023.

[12] H. Fawzi, P. Tabuada, and S. Diggavi, "Secure estimation and control for cyber-physical systems under adversarial attacks," *IEEE Transactions on Automatic Control*, vol. 59, no. 6, pp. 1454–1467, 2014.

[13] H. Zhang, Z. Li, and A. Clark, "Safe control for nonlinear systems under faults and attacks via control barrier functions," 2022. [Online]. Available: https://arxiv.org/abs/2207.05146

[14] S. Schaible, "Duality in fractional programming: A unified approach," *Operations Research*, vol. 24, no. 3, pp. 452–461, 1976.

[15] H. H. Bauschke, T. Bendit, and H. Wang, "The homogenization cone: Polar cone and projection," *Set-Valued and Variational Analysis*, vol. 31, no. 3, 2023.

[16] A. Belloni and R. M. Freund, "On the second-order feasibility cone: Primal-dual representation and efficient projection," *SIAM Journal on Optimization*, vol. 19, no. 3, pp. 1073–1092, 2008.

[17] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, no. 7, pp. 1643–1657, 2020.

[18] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, "Safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 142–11 149, 2022.

[19] F. Pasqualetti, F. Dörfler, and F. Bullo, "Attack detection and identification in cyber-physical systems," *IEEE Transactions on Automatic Control*, vol. 58, no. 11, pp. 2715–2729, 2013.

[20] F. Li and Z. Hou, "Event-triggered model-free adaptive predictive control for networked control systems under deception attacks," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 2, pp. 1325–1334, 2024.

[21] R. Hou, L. Jia, X. Bu, and J. Li, "Event-triggered model-free adaptive predictive control for networked wind-power microgrids subject to aperiodic DoS attacks," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 1737–1750, 2026.

[22] K. P. Wabersich and M. N. Zeilinger, "Linear model predictive safety certification for learning-based control," in *2018 IEEE Conference on Decision and Control*, 2018, pp. 7130–7135.

[23] A. Didier, K. P. Wabersich, and M. N. Zeilinger, "Adaptive model predictive safety certification for learning-based control," in *2021 60th IEEE Conference on Decision and Control*, 2021, pp. 809–815.

[24] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[25] L. Brunke, S. Zhou, and A. P. Schoellig, "Barrier bayesian linear regression: Online learning of control barrier conditions for safety-critical control of uncertain systems," in *Proceedings of the 4th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 168. PMLR, 2022, pp. 881–892. [Online]. Available: https://proceedings.mlr.press/v168/brunke22a.html

[26] A. Capone, R. K. Cosner, A. Ames, and S. Hirche, "Learning safe control via on-the-fly bandit exploration," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 6726–6745. [Online]. Available: https://proceedings.mlr.press/v267/capone25a.html

[27] J. Wu, H. Sibai, and Y. Vorobeychik, "Certifying safety in reinforcement learning under adversarial perturbation attacks," 2022. [Online]. Available: https://arxiv.org/abs/2212.14115

[28] L. H. Pham and J. Sun, "Certified continual learning for neural network regression," 2024. [Online]. Available: https://arxiv.org/abs/2407.06697

[29] K.-C. Hsu, D. P. Nguyen, and J. F. Fisac, "ISAACS: Iterative soft adversarial actor-critic for safety," in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 211. PMLR, 2023, pp. 90–103. [Online]. Available: https://proceedings.mlr.press/v211/hsu23a.html

[30] D. P. Nguyen, K.-C. Hsu, W. Yu, J. Tan, and J. F. Fisac, "Gameplay filters: Safe robot walking through adversarial imagination," 2024. [Online]. Available: https://arxiv.org/abs/2405.00846

[31] E. Yu, D. Žikelić, and T. A. Henzinger, "Neural control and certificate repair via runtime monitoring," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, 2025, pp. 26 409–26 417.

[32] O. Mirzaeedodangeh, E. S. Shekhtman, N. Matni, and L. Lindemann, "Safe planning in interactive environments via iterative policy updates and adversarially robust conformal prediction," in *Proceedings of The 8th Annual Learning for Dynamics and Control Conference*, ser.

Proceedings of Machine Learning Research, vol. 331. PMLR, 2026, pp. 678–704. [Online]. Available: https://proceedings.mlr.press/v331/mirzaeedodangeh26a.html

[33] Y. Ma, X. Zhang, W. Sun, and X. Zhu, "Policy poisoning in batch reinforcement learning and control," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 14 543–14 553.

[34] X. Zhang, Y. Ma, A. Singla, and X. Zhu, "Adaptive reward-poisoning attacks against reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 11 225–11 234. [Online]. Available: https://proceedings.mlr.press/v119/zhang20u.html

[35] Y. Sun, D. Huo, and F. Huang, "Vulnerability-aware poisoning mechanism for online RL with unknown dynamics," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=9r30XCjf5Dt

[36] Y. Xu, Q. Zeng, and G. Singh, "Efficient reward poisoning attacks on online deep reinforcement learning," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://openreview.net/forum?id=25G63lDHV2

[37] A. Rakhsha, G. Radanovic, R. Devidze, X. Zhu, and A. Singla, "Policy teaching via environment poisoning: Training-time adversarial attacks against reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 7974–7984. [Online]. Available: https://proceedings.mlr.press/v119/rakhsha20a.html

[38] T. Everitt, V. Krakovna, L. Orseau, and S. Legg, "Reinforcement learning with a corrupted reward channel," in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 4705–4713.

[39] L. Burbano, H. Sasahara, R. Song, Z. B. Celik, and A. A. Cárdenas, "BADControl: Backdoor attacks against control systems," in *Proceedings of the 35th USENIX Security Symposium*, 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity26/presentation/burbano

[40] A. Nika, A. Singla, and G. Radanovic, "Online defense strategies for reinforcement learning against adaptive reward poisoning," in *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 206. PMLR, 2023, pp. 335–358. [Online]. Available: https://proceedings.mlr.press/v206/nika23a.html